

Number Systems, Operations, and Codes

CHAPTER OUTLINE

- 2-1 Decimal Numbers
- 2-2 Binary Numbers
- 2-3 Decimal-to-Binary Conversion
- 2-4 Binary Arithmetic
- 2-5 Complements of Binary Numbers
- 2-6 Signed Numbers
- 2-7 Arithmetic Operations with Signed Numbers
- 2-8 Hexadecimal Numbers
- 2-9 Octal Numbers
- 2-10 Binary Coded Decimal (BCD)
- 2-11 Digital Codes
- 2-12 Error Codes

CHAPTER OBJECTIVES

- Review the decimal number system
- Count in the binary number system
- Convert from decimal to binary and from binary to decimal
- Apply arithmetic operations to binary numbers
- Determine the 1's and 2's complements of a binary number
- Express signed binary numbers in sign-magnitude, 1's complement, 2's complement, and floating-point format
- Carry out arithmetic operations with signed binary numbers
- Convert between the binary and hexadecimal number systems
- Add numbers in hexadecimal form
- Convert between the binary and octal number systems
- Express decimal numbers in binary coded decimal (BCD) form

- Add BCD numbers
- Convert between the binary system and the Gray code
- Interpret the American Standard Code for Information Interchange (ASCII)
- Explain how to detect code errors
- Discuss the cyclic redundancy check (CRC)

KEY TERMS

Key terms are in order of appearance in the chapter.

- | | |
|-------------------------|---------------------------------|
| ■ LSB | ■ BCD |
| ■ MSB | ■ Alphanumeric |
| ■ Byte | ■ ASCII |
| ■ Floating-point number | ■ Parity |
| ■ Hexadecimal | ■ Cyclic redundancy check (CRC) |
| ■ Octal | |

VISIT THE WEBSITE

Study aids for this chapter are available at <http://www.pearsonglobaleditions.com/floyd>

INTRODUCTION

The binary number system and digital codes are fundamental to computers and to digital electronics in general. In this chapter, the binary number system and its relationship to other number systems such as decimal, hexadecimal, and octal are presented. Arithmetic operations with binary numbers are covered to provide a basis for understanding how computers and many other types of digital systems work. Also, digital codes such as binary coded decimal (BCD), the Gray code, and the ASCII are covered. The parity method for detecting errors in codes is introduced. The TI-36X calculator is used to illustrate certain operations. The procedures shown may vary on other types.

2-1 Decimal Numbers

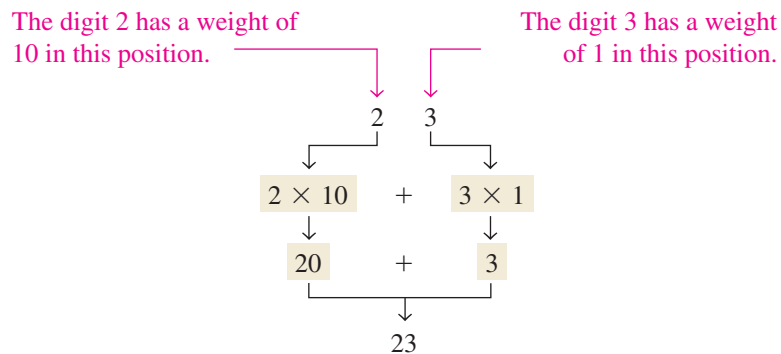
You are familiar with the decimal number system because you use decimal numbers every day. Although decimal numbers are commonplace, their weighted structure is often not understood. In this section, the structure of decimal numbers is reviewed. This review will help you more easily understand the structure of the binary number system, which is important in computers and digital electronics.

After completing this section, you should be able to

- ◆ Explain why the decimal number system is a weighted system
- ◆ Explain how powers of ten are used in the decimal system
- ◆ Determine the weight of each digit in a decimal number

The decimal number system has ten digits.

In the **decimal** number system each of the ten digits, 0 through 9, represents a certain quantity. As you know, the ten symbols (**digits**) do not limit you to expressing only ten different quantities because you use the various digits in appropriate positions within a number to indicate the magnitude of the quantity. You can express quantities up through nine before running out of digits; if you wish to express a quantity greater than nine, you use two or more digits, and the position of each digit within the number tells you the magnitude it represents. If, for example, you wish to express the quantity twenty-three, you use (by their respective positions in the number) the digit 2 to represent the quantity twenty and the digit 3 to represent the quantity three, as illustrated below.



The decimal number system has a base of 10.

The position of each digit in a decimal number indicates the magnitude of the quantity represented and can be assigned a **weight**. The weights for whole numbers are positive powers of ten that increase from right to left, beginning with $10^0 = 1$.

$$\dots 10^5 10^4 10^3 10^2 10^1 10^0$$

For fractional numbers, the weights are negative powers of ten that decrease from left to right beginning with 10^{-1} .

$$10^2 10^1 10^0 . 10^{-1} 10^{-2} 10^{-3} \dots$$

↑
Decimal point

The value of a digit is determined by its position in the number.

The value of a decimal number is the sum of the digits after each digit has been multiplied by its weight, as Examples 2-1 and 2-2 illustrate.

EXAMPLE 2-1

Express the decimal number 47 as a sum of the values of each digit.

Solution

The digit 4 has a weight of 10, which is 10^1 , as indicated by its position. The digit 7 has a weight of 1, which is 10^0 , as indicated by its position.

$$\begin{aligned} 47 &= (4 \times 10^1) + (7 \times 10^0) \\ &= (4 \times 10) + (7 \times 1) = \mathbf{40 + 7} \end{aligned}$$

Related Problem*

Determine the value of each digit in 939.

*Answers are at the end of the chapter.

EXAMPLE 2-2

Express the decimal number 568.23 as a sum of the values of each digit.

Solution

The whole number digit 5 has a weight of 100, which is 10^2 , the digit 6 has a weight of 10, which is 10^1 , the digit 8 has a weight of 1, which is 10^0 , the fractional digit 2 has a weight of 0.1, which is 10^{-1} , and the fractional digit 3 has a weight of 0.01, which is 10^{-2} .

$$\begin{aligned} 568.23 &= (5 \times 10^2) + (6 \times 10^1) + (8 \times 10^0) + (2 \times 10^{-1}) + (3 \times 10^{-2}) \\ &= (5 \times 100) + (6 \times 10) + (8 \times 1) + (2 \times 0.1) + (3 \times 0.01) \\ &= \mathbf{500 + 60 + 8 + 0.2 + 0.03} \end{aligned}$$

Related Problem

Determine the value of each digit in 67.924.

CALCULATOR SESSION**Powers of Ten**

Find the value of 10^3 .

TI-36X **Step 1:** **1** **0** **y^x**
Step 2: **3** **=**
 1000

SECTION 2-1 CHECKUP

Answers are at the end of the chapter.

- What weight does the digit 7 have in each of the following numbers?
 (a) 1370 (b) 6725 (c) 7051 (d) 58.72
- Express each of the following decimal numbers as a sum of the products obtained by multiplying each digit by its appropriate weight:
 (a) 51 (b) 137 (c) 1492 (d) 106.58

2-2 Binary Numbers

The binary number system is another way to represent quantities. It is less complicated than the decimal system because the binary system has only two digits. The decimal system with its ten digits is a base-ten system; the binary system with its two digits is a base-two system. The two binary digits (bits) are 1 and 0. The position of a 1 or 0 in a binary number indicates its weight, or value within the number, just as the position of a decimal digit determines the value of that digit. The weights in a binary number are based on powers of two.

After completing this section, you should be able to

- ♦ Count in binary
- ♦ Determine the largest decimal number that can be represented by a given number of bits
- ♦ Convert a binary number to a decimal number

Counting in Binary

The binary number system has two digits (bits).

To learn to count in the binary system, first look at how you count in the decimal system. You start at zero and count up to nine before you run out of digits. You then start another digit position (to the left) and continue counting 10 through 99. At this point you have exhausted all two-digit combinations, so a third digit position is needed to count from 100 through 999.

A comparable situation occurs when you count in binary, except that you have only two digits, called *bits*. Begin counting: 0, 1. At this point you have used both digits, so include another digit position and continue: 10, 11. You have now exhausted all combinations of two digits, so a third position is required. With three digit positions you can continue to count: 100, 101, 110, and 111. Now you need a fourth digit position to continue, and so on. A binary count of zero through fifteen is shown in Table 2–1. Notice the patterns with which the 1s and 0s alternate in each column.

The binary number system has a base of 2.

InfoNote

In processor operations, there are many cases where adding or subtracting 1 to a number stored in a counter is necessary. Processors have special instructions that use less time and generate less machine code than the ADD or SUB instructions. For the Intel processors, the INC (increment) instruction adds 1 to a number. For subtraction, the corresponding instruction is DEC (decrement), which subtracts 1 from a number.

TABLE 2-1

Decimal Number	Binary Number			
0	0	0	0	0
1	0	0	0	1
2	0	0	1	0
3	0	0	1	1
4	0	1	0	0
5	0	1	0	1
6	0	1	1	0
7	0	1	1	1
8	1	0	0	0
9	1	0	0	1
10	1	0	1	0
11	1	0	1	1
12	1	1	0	0
13	1	1	0	1
14	1	1	1	0
15	1	1	1	1

The value of a bit is determined by its position in the number.

CALCULATOR SESSION

Powers of Two

Find the value of 2^5 .

TI-36X Step 1: **2** **y^x**
Step 2: **5** **=**
32

As you have seen in Table 2–1, four bits are required to count from zero to 15. In general, with n bits you can count up to a number equal to $2^n - 1$.

$$\text{Largest decimal number} = 2^n - 1$$

For example, with five bits ($n = 5$) you can count from zero to thirty-one.

$$2^5 - 1 = 32 - 1 = 31$$

With six bits ($n = 6$) you can count from zero to sixty-three.

$$2^6 - 1 = 64 - 1 = 63$$

An Application

Learning to count in binary will help you to basically understand how digital circuits can be used to count events. Let's take a simple example of counting tennis balls going into a box from a conveyor belt. Assume that nine balls are to go into each box.

The counter in Figure 2-1 counts the pulses from a sensor that detects the passing of a ball and produces a sequence of logic levels (digital waveforms) on each of its four parallel outputs. Each set of logic levels represents a 4-bit binary number (HIGH = 1 and LOW = 0), as indicated. As the decoder receives these waveforms, it decodes each set of four bits and converts it to the corresponding decimal number in the 7-segment display. When the counter gets to the binary state of 1001, it has counted nine tennis balls, the display shows decimal 9, and a new box is moved under the conveyor belt. Then the counter goes back to its zero state (0000), and the process starts over. (The number 9 was used only in the interest of single-digit simplicity.)

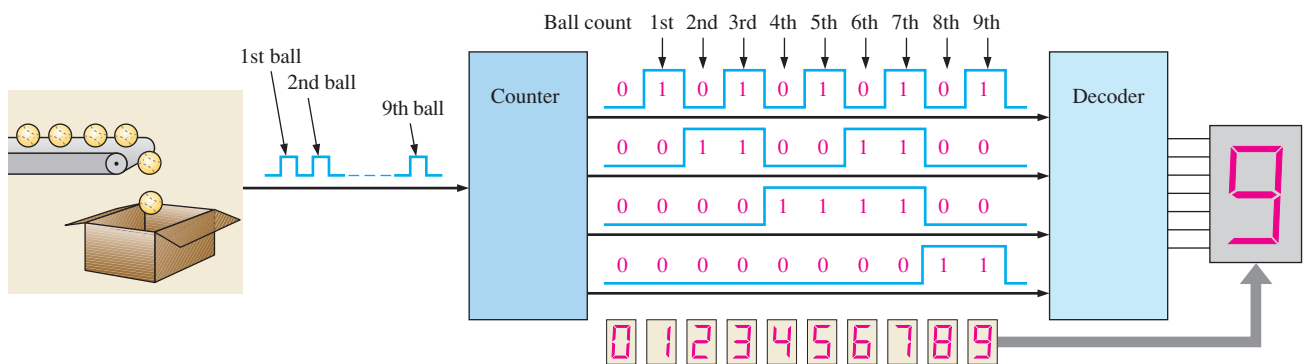


FIGURE 2-1 Illustration of a simple binary counting application.

The Weighting Structure of Binary Numbers

A binary number is a weighted number. The right-most bit is the **LSB** (least significant bit) in a binary whole number and has a weight of $2^0 = 1$. The weights increase from right to left by a power of two for each bit. The left-most bit is the **MSB** (most significant bit); its weight depends on the size of the binary number.

Fractional numbers can also be represented in binary by placing bits to the right of the binary point, just as fractional decimal digits are placed to the right of the decimal point. The left-most bit is the MSB in a binary fractional number and has a weight of $2^{-1} = 0.5$. The fractional weights decrease from left to right by a negative power of two for each bit.

The weight structure of a binary number is

$$2^{n-1} \dots 2^3 2^2 2^1 2^0 \cdot 2^{-1} 2^{-2} \dots 2^{-n}$$

↑
Binary point

where n is the number of bits from the binary point. Thus, all the bits to the left of the binary point have weights that are positive powers of two, as previously discussed for whole numbers. All bits to the right of the binary point have weights that are negative powers of two, or fractional weights.

The powers of two and their equivalent decimal weights for an 8-bit binary whole number and a 6-bit binary fractional number are shown in Table 2-2. Notice that the weight doubles for each positive power of two and that the weight is halved for each negative power of two. You can easily extend the table by doubling the weight of the most significant positive power of two and halving the weight of the least significant negative power of two; for example, $2^9 = 512$ and $2^{-7} = 0.0078125$.

The weight or value of a bit increases from right to left in a binary number.

InfoNote

Processors use binary numbers to select memory locations. Each location is assigned a unique number called an *address*. Some microprocessors, for example, have 32 address lines which can select 2^{32} (4,294,967,296) unique locations.

TABLE 2-2

Binary weights.

Positive Powers of Two (Whole Numbers)									Negative Powers of Two (Fractional Number)					
2^8	2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0	2^{-1}	2^{-2}	2^{-3}	2^{-4}	2^{-5}	2^{-6}
256	128	64	32	16	8	4	2	1	1/2	1/4	1/8	1/16	1/32	1/64
									0.5	0.25	0.125	0.625	0.03125	0.015625

Binary-to-Decimal Conversion

Add the weights of all 1s in a binary number to get the decimal value.

The decimal value of any binary number can be found by adding the weights of all bits that are 1 and discarding the weights of all bits that are 0.

EXAMPLE 2-3

Convert the binary whole number 1101101 to decimal.

Solution

Determine the weight of each bit that is a 1, and then find the sum of the weights to get the decimal number.

$$\begin{aligned}
 \text{Weight: } & 2^6 \ 2^5 \ 2^4 \ 2^3 \ 2^2 \ 2^1 \ 2^0 \\
 \text{Binary number: } & 1 \ 1 \ 0 \ 1 \ 1 \ 0 \ 1 \\
 1101101 = & 2^6 + 2^5 + 2^3 + 2^2 + 2^0 \\
 = & 64 + 32 + 8 + 4 + 1 = \mathbf{109}
 \end{aligned}$$

Related Problem

Convert the binary number 10010001 to decimal.

EXAMPLE 2-4

Convert the fractional binary number 0.1011 to decimal.

Solution

Determine the weight of each bit that is a 1, and then sum the weights to get the decimal fraction.

$$\begin{aligned}
 \text{Weight: } & 2^{-1} \ 2^{-2} \ 2^{-3} \ 2^{-4} \\
 \text{Binary number: } & 0.1 \ 0 \ 1 \ 1 \\
 0.1011 = & 2^{-1} + 2^{-3} + 2^{-4} \\
 = & 0.5 + 0.125 + 0.0625 = \mathbf{0.6875}
 \end{aligned}$$

Related Problem

Convert the binary number 10.111 to decimal.

SECTION 2-2 CHECKUP

1. What is the largest decimal number that can be represented in binary with eight bits?
2. Determine the weight of the 1 in the binary number 10000.
3. Convert the binary number 10111101.011 to decimal.

2-3 Decimal-to-Binary Conversion

In Section 2-2 you learned how to convert a binary number to the equivalent decimal number. Now you will learn two ways of converting from a decimal number to a binary number.

After completing this section, you should be able to

- ♦ Convert a decimal number to binary using the sum-of-weights method
- ♦ Convert a decimal whole number to binary using the repeated division-by-2 method
- ♦ Convert a decimal fraction to binary using the repeated multiplication-by-2 method

Sum-of-Weights Method

One way to find the binary number that is equivalent to a given decimal number is to determine the set of binary weights whose sum is equal to the decimal number. An easy way to remember binary weights is that the lowest is 1, which is 2^0 , and that by doubling any weight, you get the next higher weight; thus, a list of seven binary weights would be 64, 32, 16, 8, 4, 2, 1 as you learned in the last section. The decimal number 9, for example, can be expressed as the sum of binary weights as follows:

$$9 = 8 + 1 \quad \text{or} \quad 9 = 2^3 + 2^0$$

Placing 1s in the appropriate weight positions, 2^3 and 2^0 , and 0s in the 2^2 and 2^1 positions determines the binary number for decimal 9.

$$\begin{array}{cccc} 2^3 & 2^2 & 2^1 & 2^0 \\ 1 & 0 & 0 & 1 \end{array} \quad \text{Binary number for decimal 9}$$

To get the binary number for a given decimal number, find the binary weights that add up to the decimal number.

EXAMPLE 2-5

Convert the following decimal numbers to binary:

- (a) 12 (b) 25
(c) 58 (d) 82

Solution

- (a) $12 = 8 + 4 = 2^3 + 2^2 \longrightarrow 1100$
 (b) $25 = 16 + 8 + 1 = 2^4 + 2^3 + 2^0 \longrightarrow 11001$
 (c) $58 = 32 + 16 + 8 + 2 = 2^5 + 2^4 + 2^3 + 2^1 \longrightarrow 111010$
 (d) $82 = 64 + 16 + 2 = 2^6 + 2^4 + 2^1 \longrightarrow 1010010$

Related Problem

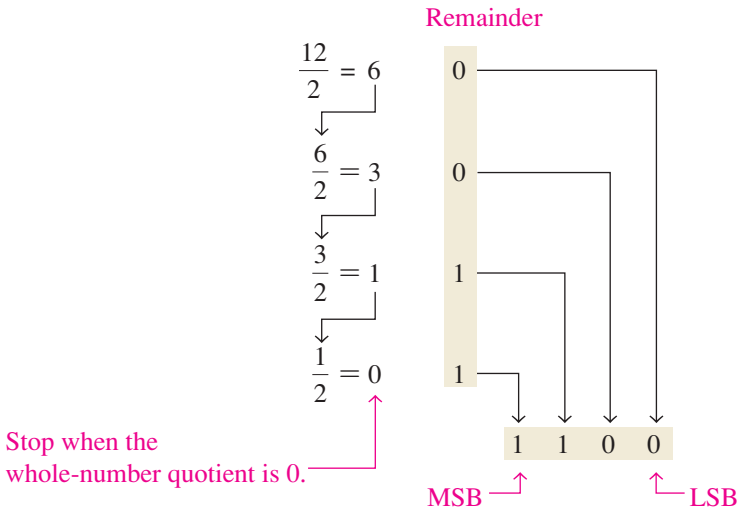
Convert the decimal number 125 to binary.

Repeated Division-by-2 Method

A systematic method of converting whole numbers from decimal to binary is the *repeated division-by-2* process. For example, to convert the decimal number 12 to binary, begin by dividing 12 by 2. Then divide each resulting quotient by 2 until there is a 0 whole-number quotient. The **remainders** generated by each division form the binary number. The first remainder to be produced is the LSB (least significant bit) in the binary number, and the

To get the binary number for a given decimal number, divide the decimal number by 2 until the quotient is 0. Remainders form the binary number.

last remainder to be produced is the MSB (most significant bit). This procedure is illustrated as follows for converting the decimal number 12 to binary.

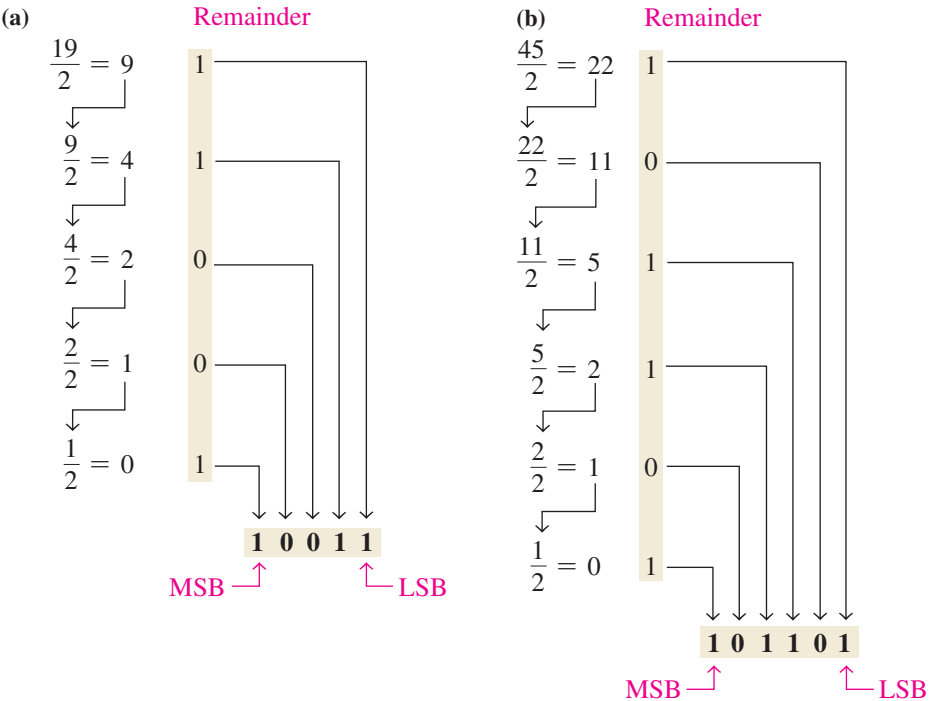


EXAMPLE 2-6

Convert the following decimal numbers to binary:

- (a) 19 (b) 45

Solution



CALCULATOR SESSION

Conversion of a Decimal Number to a Binary Number
Convert decimal 57 to binary.

TI-36X Step 1: 3rd EE DEC

Step 2: 5 7 BIN

Step 3: 3rd X

111001

Related Problem

Convert decimal number 39 to binary.

Converting Decimal Fractions to Binary

Examples 2–5 and 2–6 demonstrated whole-number conversions. Now let's look at fractional conversions. An easy way to remember fractional binary weights is that the most significant weight is 0.5, which is 2^{-1} , and that by halving any weight, you get the next lower weight; thus a list of four fractional binary weights would be 0.5, 0.25, 0.125, 0.0625.

Sum-of-Weights

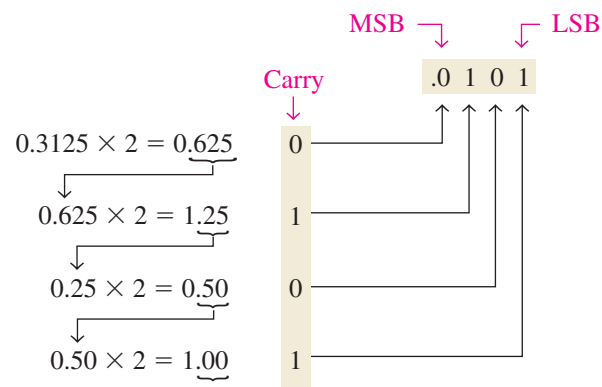
The sum-of-weights method can be applied to fractional decimal numbers, as shown in the following example:

$$0.625 = 0.5 + 0.125 = 2^{-1} + 2^{-3} = 0.101$$

There is a 1 in the 2^{-1} position, a 0 in the 2^{-2} position, and a 1 in the 2^{-3} position.

Repeated Multiplication by 2

As you have seen, decimal whole numbers can be converted to binary by repeated division by 2. Decimal fractions can be converted to binary by repeated multiplication by 2. For example, to convert the decimal fraction 0.3125 to binary, begin by multiplying 0.3125 by 2 and then multiplying each resulting fractional part of the product by 2 until the fractional product is zero or until the desired number of decimal places is reached. The carry digits, or **carries**, generated by the multiplications produce the binary number. The first carry produced is the MSB, and the last carry is the LSB. This procedure is illustrated as follows:



Continue to the desired number of decimal places or stop when the fractional part is all zeros.

SECTION 2-3 CHECKUP

- Convert each decimal number to binary by using the sum-of-weights method:
 - 23
 - 57
 - 45.5
- Convert each decimal number to binary by using the repeated division-by-2 method (repeated multiplication-by-2 for fractions):
 - 14
 - 21
 - 0.375

2-4 Binary Arithmetic

Binary arithmetic is essential in all digital computers and in many other types of digital systems. To understand digital systems, you must know the basics of binary addition, subtraction, multiplication, and division. This section provides an introduction that will be expanded in later sections.

After completing this section, you should be able to

- ◆ Add binary numbers
- ◆ Subtract binary numbers
- ◆ Multiply binary numbers
- ◆ Divide binary numbers

Binary Addition

In binary $1 + 1 = 10$, not 2.

The four basic rules for adding binary digits (bits) are as follows:

$0 + 0 = 0$	Sum of 0 with a carry of 0
$0 + 1 = 1$	Sum of 1 with a carry of 0
$1 + 0 = 1$	Sum of 1 with a carry of 0
$1 + 1 = 10$	Sum of 0 with a carry of 1

Notice that the first three rules result in a single bit and in the fourth rule the addition of two 1s yields a binary two (10). When binary numbers are added, the last condition creates a sum of 0 in a given column and a carry of 1 over to the next column to the left, as illustrated in the following addition of $11 + 1$:

	Carry	Carry	
	1	1	
	←	←	
0	1	1	
+ 0	0	1	
1	0	0	

In the right column, $1 + 1 = 0$ with a carry of 1 to the next column to the left. In the middle column, $1 + 1 + 0 = 0$ with a carry of 1 to the next column to the left. In the left column, $1 + 0 + 0 = 1$.

When there is a carry of 1, you have a situation in which three bits are being added (a bit in each of the two numbers and a carry bit). This situation is illustrated as follows:

Carry bits	↓	
1	$+ 0 + 0 = 01$	Sum of 1 with a carry of 0
1	$+ 1 + 0 = 10$	Sum of 0 with a carry of 1
1	$+ 0 + 1 = 10$	Sum of 0 with a carry of 1
1	$+ 1 + 1 = 11$	Sum of 1 with a carry of 1

EXAMPLE 2-7

Add the following binary numbers:

- (a) $11 + 11$ (b) $100 + 10$
(c) $111 + 11$ (d) $110 + 100$

Solution

The equivalent decimal addition is also shown for reference.

$$\begin{array}{r} \text{(a)} \quad 11 \quad 3 \\ + 11 \quad + 3 \\ \hline 110 \quad 6 \end{array} \quad \begin{array}{r} \text{(b)} \quad 100 \quad 4 \\ + 10 \quad + 2 \\ \hline 110 \quad 6 \end{array}$$

$$\begin{array}{r} \text{(c)} \quad 111 \quad 7 \\ + 11 \quad + 3 \\ \hline 1010 \quad 10 \end{array} \quad \begin{array}{r} \text{(d)} \quad 110 \quad 6 \\ + 100 \quad + 4 \\ \hline 1010 \quad 10 \end{array}$$

Related Problem

Add 1111 and 1100.

Binary Subtraction

The four basic rules for subtracting bits are as follows:

In binary $10 - 1 = 1$, not 9.

$$\begin{array}{l} 0 - 0 = 0 \\ 1 - 1 = 0 \\ 1 - 0 = 1 \\ 10 - 1 = 1 \quad 0 - 1 \text{ with a borrow of } 1 \end{array}$$

When subtracting numbers, you sometimes have to borrow from the next column to the left. A borrow is required in binary only when you try to subtract a 1 from a 0. In this case, when a 1 is borrowed from the next column to the left, a 10 is created in the column being subtracted, and the last of the four basic rules just listed must be applied. Examples 2–8 and 2–9 illustrate binary subtraction; the equivalent decimal subtractions are also shown.

EXAMPLE 2–8

Perform the following binary subtractions:

$$\text{(a)} \quad 11 - 01 \quad \text{(b)} \quad 11 - 10$$

Solution

$$\begin{array}{r} \text{(a)} \quad 11 \quad 3 \\ - 01 \quad - 1 \\ \hline 10 \quad 2 \end{array} \quad \begin{array}{r} \text{(b)} \quad 11 \quad 3 \\ - 10 \quad - 2 \\ \hline 01 \quad 1 \end{array}$$

No borrows were required in this example. The binary number 01 is the same as 1.

Related Problem

Subtract 100 from 111.

EXAMPLE 2–9

Subtract 011 from 101.

Solution

$$\begin{array}{r} 101 \quad 5 \\ - 011 \quad - 3 \\ \hline 010 \quad 2 \end{array}$$

Let's examine exactly what was done to subtract the two binary numbers since a borrow is required. Begin with the right column.

Left column:
When a 1 is borrowed,
a 0 is left, so $0 - 0 = 0$.

Middle column:
Borrow 1 from next column
to the left, making a 10 in
this column, then $10 - 1 = 1$.

Right column:
 $1 - 1 = 0$

0101
-011

010

Related Problem
Subtract 101 from 110.

Binary multiplication of two bits is the same as multiplication of the decimal digits 0 and 1.

Binary Multiplication

The four basic rules for multiplying bits are as follows:

- $0 \times 0 = 0$
- $0 \times 1 = 0$
- $1 \times 0 = 0$
- $1 \times 1 = 1$

Multiplication is performed with binary numbers in the same manner as with decimal numbers. It involves forming partial products, shifting each successive partial product left one place, and then adding all the partial products. Example 2-10 illustrates the procedure; the equivalent decimal multiplications are shown for reference.

EXAMPLE 2-10

Perform the following binary multiplications:
(a) 11×11 (b) 101×111

Solution

(a)

11

$\times 11$

11

$+11$

1001

Partial products {

(b)

111

$\times 101$

111

000

$+111$

100011

Partial products {

7

$\times 5$

35

Related Problem

Multiply 1101×1010 .

A calculator can be used to perform arithmetic operations with binary numbers as long as the capacity of the calculator is not exceeded.

Binary Division

Division in binary follows the same procedure as division in decimal, as Example 2-11 illustrates. The equivalent decimal divisions are also given.

EXAMPLE 2-11

Perform the following binary divisions:
(a) $110 \div 11$ (b) $110 \div 10$

Solution

$\begin{array}{r} 10 \\ 11 \overline{)110} \\ \underline{11} \\ 000 \end{array}$	$\begin{array}{r} 2 \\ 3 \overline{)6} \\ \underline{6} \\ 0 \end{array}$	$\begin{array}{r} 11 \quad 3 \\ 10 \overline{)110} \quad 2 \overline{)6} \\ \underline{10} \quad \underline{6} \\ 10 \quad 0 \\ \underline{10} \\ 00 \end{array}$
--	---	---

Related Problem

Divide 1100 by 100.

SECTION 2-4 CHECKUP

1. Perform the following binary additions:

(a) $1101 + 1010$ (b) $10111 + 01101$

2. Perform the following binary subtractions:

(a) $1101 - 0100$ (b) $1001 - 0111$

3. Perform the indicated binary operations:

(a) 110×111 (b) $1100 \div 011$

2-5 Complements of Binary Numbers

The 1's complement and the 2's complement of a binary number are important because they permit the representation of negative numbers. The method of 2's complement arithmetic is commonly used in computers to handle negative numbers.

After completing this section, you should be able to

- ♦ Convert a binary number to its 1's complement
- ♦ Convert a binary number to its 2's complement using either of two methods

Finding the 1's Complement

The 1's **complement** of a binary number is found by changing all 1s to 0s and all 0s to 1s, as illustrated below:

Change each bit in a number to get the 1's complement.

1 0 1 1 0 0 1 0	Binary number
↓ ↓ ↓ ↓ ↓ ↓ ↓ ↓	
0 1 0 0 1 1 0 1	1's complement

The simplest way to obtain the 1's complement of a binary number with a digital circuit is to use parallel inverters (NOT circuits), as shown in Figure 2-2 for an 8-bit binary number.

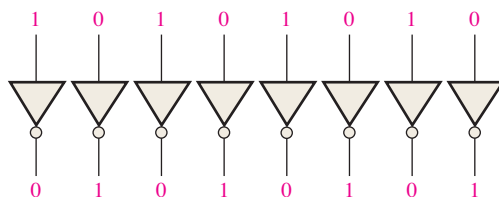


FIGURE 2-2 Example of inverters used to obtain the 1's complement of a binary number.

Finding the 2's Complement

Add 1 to the 1's complement to get the 2's complement.

The 2's complement of a binary number is found by adding 1 to the LSB of the 1's complement.
 $2's\ complement = (1's\ complement) + 1$

EXAMPLE 2-12

Find the 2's complement of 10110010.

Solution

10110010	Binary number
01001101	1's complement
+ 1	Add 1
01001110	2's complement

Related Problem

Determine the 2's complement of 11001011.

Change all bits to the left of the least significant 1 to get 2's complement.

- An alternative method of finding the 2's complement of a binary number is as follows:
1. Start at the right with the LSB and write the bits as they are up to and including the first 1.
 2. Take the 1's complements of the remaining bits.

EXAMPLE 2-13

Find the 2's complement of 10111000 using the alternative method.

Solution

	10111000	Binary number
1's complements of original bits	01001000	2's complement

These bits stay the same.

Related Problem

Find the 2's complement of 11000000.

The 2's complement of a negative binary number can be realized using inverters and an adder, as indicated in Figure 2-3. This illustrates how an 8-bit number can be converted to its 2's complement by first inverting each bit (taking the 1's complement) and then adding 1 to the 1's complement with an adder circuit.

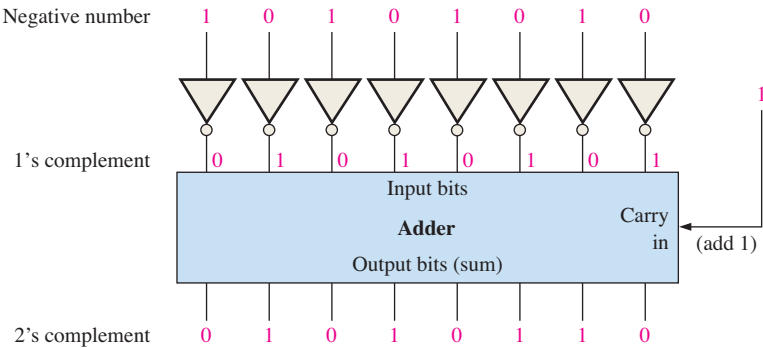


FIGURE 2-3 Example of obtaining the 2's complement of a negative binary number.

To convert from a 1's or 2's complement back to the true (uncomplemented) binary form, use the same two procedures described previously. To go from the 1's complement back to true binary, reverse all the bits. To go from the 2's complement form back to true binary, take the 1's complement of the 2's complement number and add 1 to the least significant bit.

SECTION 2-5 CHECKUP

1. Determine the 1's complement of each binary number:

(a) 00011010 (b) 11110111 (c) 10001101

2. Determine the 2's complement of each binary number:

(a) 00010110 (b) 11111100 (c) 10010001

2-6 Signed Numbers

Digital systems, such as the computer, must be able to handle both positive and negative numbers. A signed binary number consists of both sign and magnitude information. The sign indicates whether a number is positive or negative, and the magnitude is the value of the number. There are three forms in which signed integer (whole) numbers can be represented in binary: sign-magnitude, 1's complement, and 2's complement. Of these, the 2's complement is the most important and the sign-magnitude is the least used. Noninteger and very large or small numbers can be expressed in floating-point format.

After completing this section, you should be able to

- ◆ Express positive and negative numbers in sign-magnitude
- ◆ Express positive and negative numbers in 1's complement
- ◆ Express positive and negative numbers in 2's complement
- ◆ Determine the decimal value of signed binary numbers
- ◆ Express a binary number in floating-point format

The Sign Bit

The left-most bit in a signed binary number is the **sign bit**, which tells you whether the number is positive or negative.

A 0 sign bit indicates a positive number, and a 1 sign bit indicates a negative number.

Sign-Magnitude Form

When a signed binary number is represented in sign-magnitude, the left-most bit is the sign bit and the remaining bits are the magnitude bits. The magnitude bits are in true (uncomplemented) binary for both positive and negative numbers. For example, the decimal number +25 is expressed as an 8-bit signed binary number using the sign-magnitude form as

00011001
 Sign bit ——— ↑ ↑ ——— Magnitude bits

The decimal number −25 is expressed as

10011001

Notice that the only difference between +25 and −25 is the sign bit because the magnitude bits are in true binary for both positive and negative numbers.

In the sign-magnitude form, a negative number has the same magnitude bits as the corresponding positive number but the sign bit is a 1 rather than a zero.

InfoNote

Processors use the 2's complement for negative integer numbers in arithmetic operations. The reason is that subtraction of a number is the same as adding the 2's complement of the number. Processors form the 2's complement by inverting the bits and adding 1, using special instructions that produce the same result as the adder in Figure 2-3.

1's Complement Form

Positive numbers in 1's complement form are represented the same way as the positive sign-magnitude numbers. Negative numbers, however, are the 1's complements of the corresponding positive numbers. For example, using eight bits, the decimal number -25 is expressed as the 1's complement of $+25$ (00011001) as

$$11100110$$

In the 1's complement form, a negative number is the 1's complement of the corresponding positive number.

2's Complement Form

Positive numbers in 2's complement form are represented the same way as in the sign-magnitude and 1's complement forms. Negative numbers are the 2's complements of the corresponding positive numbers. Again, using eight bits, let's take decimal number -25 and express it as the 2's complement of $+25$ (00011001). Inverting each bit and adding 1, you get

$$-25 = 11100111$$

In the 2's complement form, a negative number is the 2's complement of the corresponding positive number.

EXAMPLE 2-14

Express the decimal number -39 as an 8-bit number in the sign-magnitude, 1's complement, and 2's complement forms.

Solution

First, write the 8-bit number for $+39$.

$$00100111$$

In the *sign-magnitude form*, -39 is produced by changing the sign bit to a 1 and leaving the magnitude bits as they are. The number is

$$10100111$$

In the *1's complement form*, -39 is produced by taking the 1's complement of $+39$ (00100111).

$$11011000$$

In the *2's complement form*, -39 is produced by taking the 2's complement of $+39$ (00100111) as follows:

$$\begin{array}{r} 11011000 \quad \text{1's complement} \\ + \quad \quad 1 \\ \hline 11011001 \quad \text{2's complement} \end{array}$$

Related Problem

Express $+19$ and -19 as 8-bit numbers in sign-magnitude, 1's complement, and 2's complement.

The Decimal Value of Signed Numbers**Sign-Magnitude**

Decimal values of positive and negative numbers in the sign-magnitude form are determined by summing the weights in all the magnitude bit positions where there are 1s and ignoring those positions where there are zeros. The sign is determined by examination of the sign bit.

EXAMPLE 2-15

Determine the decimal value of this signed binary number expressed in sign-magnitude: 10010101.

Solution

The seven magnitude bits and their powers-of-two weights are as follows:

$$\begin{array}{ccccccc} 2^6 & 2^5 & 2^4 & 2^3 & 2^2 & 2^1 & 2^0 \\ 0 & 0 & 1 & 0 & 1 & 0 & 1 \end{array}$$

Summing the weights where there are 1s,

$$16 + 4 + 1 = 21$$

The sign bit is 1; therefore, the decimal number is **-21**.

Related Problem

Determine the decimal value of the sign-magnitude number 01110111.

1's Complement

Decimal values of positive numbers in the 1's complement form are determined by summing the weights in all bit positions where there are 1s and ignoring those positions where there are zeros. Decimal values of negative numbers are determined by assigning a negative value to the weight of the sign bit, summing all the weights where there are 1s, and adding 1 to the result.

EXAMPLE 2-16

Determine the decimal values of the signed binary numbers expressed in 1's complement:

(a) 00010111 (b) 11101000

Solution

(a) The bits and their powers-of-two weights for the positive number are as follows:

$$\begin{array}{ccccccc} -2^7 & 2^6 & 2^5 & 2^4 & 2^3 & 2^2 & 2^1 & 2^0 \\ 0 & 0 & 0 & 1 & 0 & 1 & 1 & 1 \end{array}$$

Summing the weights where there are 1s,

$$16 + 4 + 2 + 1 = \mathbf{+23}$$

(b) The bits and their powers-of-two weights for the negative number are as follows. Notice that the negative sign bit has a weight of -2^7 or -128 .

$$\begin{array}{ccccccc} -2^7 & 2^6 & 2^5 & 2^4 & 2^3 & 2^2 & 2^1 & 2^0 \\ 1 & 1 & 1 & 0 & 1 & 0 & 0 & 0 \end{array}$$

Summing the weights where there are 1s,

$$-128 + 64 + 32 + 8 = -24$$

Adding 1 to the result, the final decimal number is

$$-24 + 1 = \mathbf{-23}$$

Related Problem

Determine the decimal value of the 1's complement number 11101011.

2's Complement

Decimal values of positive and negative numbers in the 2's complement form are determined by summing the weights in all bit positions where there are 1s and ignoring those positions where there are zeros. The weight of the sign bit in a negative number is given a negative value.

EXAMPLE 2-17

Determine the decimal values of the signed binary numbers expressed in 2's complement:

- (a) 01010110 (b) 10101010

Solution

- (a) The bits and their powers-of-two weights for the positive number are as follows:

$$\begin{array}{cccccccc} -2^7 & 2^6 & 2^5 & 2^4 & 2^3 & 2^2 & 2^1 & 2^0 \\ 0 & 1 & 0 & 1 & 0 & 1 & 1 & 0 \end{array}$$

Summing the weights where there are 1s,

$$64 + 16 + 4 + 2 = +86$$

- (b) The bits and their powers-of-two weights for the negative number are as follows. Notice that the negative sign bit has a weight of $-2^7 = -128$.

$$\begin{array}{cccccccc} -2^7 & 2^6 & 2^5 & 2^4 & 2^3 & 2^2 & 2^1 & 2^0 \\ 1 & 0 & 1 & 0 & 1 & 0 & 1 & 0 \end{array}$$

Summing the weights where there are 1s,

$$-128 + 32 + 8 + 2 = -86$$

Related Problem

Determine the decimal value of the 2's complement number 11010111.

From these examples, you can see why the 2's complement form is preferred for representing signed integer numbers: To convert to decimal, it simply requires a summation of weights regardless of whether the number is positive or negative. The 1's complement system requires adding 1 to the summation of weights for negative numbers but not for positive numbers. Also, the 1's complement form is generally not used because two representations of zero (00000000 or 11111111) are possible.

Range of Signed Integer Numbers

We have used 8-bit numbers for illustration because the 8-bit grouping is common in most computers and has been given the special name **byte**. With one byte or eight bits, you can represent 256 different numbers. With two bytes or sixteen bits, you can represent 65,536 different numbers. With four bytes or 32 bits, you can represent 4.295×10^9 different numbers. The formula for finding the number of different combinations of n bits is

$$\text{Total combinations} = 2^n$$

For 2's complement signed numbers, the range of values for n -bit numbers is

$$\text{Range} = -(2^{n-1}) \text{ to } +(2^{n-1} - 1)$$

where in each case there is one sign bit and $n - 1$ magnitude bits. For example, with four bits you can represent numbers in 2's complement ranging from $-(2^3) = -8$ to $2^3 - 1 = +7$. Similarly, with eight bits you can go from -128 to $+127$, with sixteen bits you can go from

The range of magnitude values represented by binary numbers depends on the number of bits (n).

−32,768 to +32,767, and so on. There is one less positive number than there are negative numbers because zero is represented as a positive number (all zeros).

Floating-Point Numbers

To represent very large **integer** (whole) numbers, many bits are required. There is also a problem when numbers with both integer and fractional parts, such as 23.5618, need to be represented. The floating-point number system, based on scientific notation, is capable of representing very large and very small numbers without an increase in the number of bits and also for representing numbers that have both integer and fractional components.

A **floating-point number** (also known as a *real number*) consists of two parts plus a sign. The **mantissa** is the part of a floating-point number that represents the magnitude of the number and is between 0 and 1. The **exponent** is the part of a floating-point number that represents the number of places that the decimal point (or binary point) is to be moved.

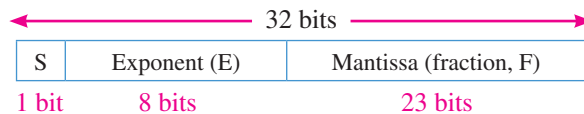
A decimal example will be helpful in understanding the basic concept of floating-point numbers. Let's consider a decimal number which, in integer form, is 241,506,800. The mantissa is .2415068 and the exponent is 9. When the integer is expressed as a floating-point number, it is normalized by moving the decimal point to the left of all the digits so that the mantissa is a fractional number and the exponent is the power of ten. The floating-point number is written as

$$0.2415068 \times 10^9$$

For binary floating-point numbers, the format is defined by ANSI/IEEE Standard 754-1985 in three forms: *single-precision*, *double-precision*, and *extended-precision*. These all have the same basic formats except for the number of bits. Single-precision floating-point numbers have 32 bits, double-precision numbers have 64 bits, and extended-precision numbers have 80 bits. We will restrict our discussion to the single-precision floating-point format.

Single-Precision Floating-Point Binary Numbers

In the standard format for a single-precision binary number, the sign bit (S) is the left-most bit, the exponent (E) includes the next eight bits, and the mantissa or fractional part (F) includes the remaining 23 bits, as shown next.



In the mantissa or fractional part, the binary point is understood to be to the left of the 23 bits. Effectively, there are 24 bits in the mantissa because in any binary number the left-most (most significant) bit is always a 1. Therefore, this 1 is understood to be there although it does not occupy an actual bit position.

The eight bits in the exponent represent a *biased exponent*, which is obtained by adding 127 to the actual exponent. The purpose of the bias is to allow very large or very small numbers without requiring a separate sign bit for the exponents. The biased exponent allows a range of actual exponent values from −126 to +128.

To illustrate how a binary number is expressed in floating-point format, let's use 1011010010001 as an example. First, it can be expressed as 1 plus a fractional binary number by moving the binary point 12 places to the left and then multiplying by the appropriate power of two.

$$1011010010001 = 1.011010010001 \times 2^{12}$$

Assuming that this is a positive number, the sign bit (S) is 0. The exponent, 12, is expressed as a biased exponent by adding it to 127 ($12 + 127 = 139$). The biased exponent (E) is expressed as the binary number 10001011. The mantissa is the fractional part (F) of the binary number, .011010010001. Because there is always a 1 to the left of the binary point

InfoNote

In addition to the CPU (central processing unit), computers use *coprocessors* to perform complicated mathematical calculations using floating-point numbers. The purpose is to increase performance by freeing up the CPU for other tasks. The mathematical coprocessor is also known as the floating-point unit (FPU).

in the power-of-two expression, it is not included in the mantissa. The complete floating-point number is

S	E	F
0	10001011	011010010001000000000000

Next, let's see how to evaluate a binary number that is already in floating-point format. The general approach to determining the value of a floating-point number is expressed by the following formula:

$$\text{Number} = (-1)^S(1 + F)(2^{E-127})$$

To illustrate, let's consider the following floating-point binary number:

S	E	F
1	10010001	100011100010000000000000

The sign bit is 1. The biased exponent is $10010001 = 145$. Applying the formula, we get

$$\begin{aligned}\text{Number} &= (-1)^1(1.10001110001)(2^{145-127}) \\ &= (-1)(1.10001110001)(2^{18}) = -11000111000100000000\end{aligned}$$

This floating-point binary number is equivalent to $-407,688$ in decimal. Since the exponent can be any number between -126 and $+128$, extremely large and small numbers can be expressed. A 32-bit floating-point number can replace a binary integer number having 129 bits. Because the exponent determines the position of the binary point, numbers containing both integer and fractional parts can be represented.

There are two exceptions to the format for floating-point numbers: The number 0.0 is represented by all 0s, and infinity is represented by all 1s in the exponent and all 0s in the mantissa.

EXAMPLE 2-18

Convert the decimal number 3.248×10^4 to a single-precision floating-point binary number.

Solution

Convert the decimal number to binary.

$$3.248 \times 10^4 = 32480 = 111111011100000_2 = 1.11111011100000 \times 2^{14}$$

The MSB will not occupy a bit position because it is always a 1. Therefore, the mantissa is the fractional 23-bit binary number 11111011100000000000000 and the biased exponent is

$$14 + 127 = 141 = 10001101_2$$

The complete floating-point number is

0	10001101	11111011100000000000000
---	----------	-------------------------

Related Problem

Determine the binary value of the following floating-point binary number:

0 10011000 10000100010100110000000

SECTION 2-6 CHECKUP

- Express the decimal number $+9$ as an 8-bit binary number in the sign-magnitude system.
- Express the decimal number -33 as an 8-bit binary number in the 1's complement system.
- Express the decimal number -46 as an 8-bit binary number in the 2's complement system.
- List the three parts of a signed, floating-point number.

2-7 Arithmetic Operations with Signed Numbers

In the last section, you learned how signed numbers are represented in three different forms. In this section, you will learn how signed numbers are added, subtracted, multiplied, and divided. Because the 2's complement form for representing signed numbers is the most widely used in computers and microprocessor-based systems, the coverage in this section is limited to 2's complement arithmetic. The processes covered can be extended to the other forms if necessary.

After completing this section, you should be able to

- ◆ Add signed binary numbers
- ◆ Define *overflow*
- ◆ Explain how computers add strings of numbers
- ◆ Subtract signed binary numbers
- ◆ Multiply signed binary numbers using the direct addition method
- ◆ Multiply signed binary numbers using the partial products method
- ◆ Divide signed binary numbers

Addition

The two numbers in an addition are the **addend** and the **augend**. The result is the **sum**. There are four cases that can occur when two signed binary numbers are added.

1. Both numbers positive
2. Positive number with magnitude larger than negative number
3. Negative number with magnitude larger than positive number
4. Both numbers negative

Let's take one case at a time using 8-bit signed numbers as examples. The equivalent decimal numbers are shown for reference.

Both numbers positive:

$$\begin{array}{r} 00001111 \quad 7 \\ + 00001000 \quad + 4 \\ \hline 00010111 \quad 11 \end{array}$$

Addition of two positive numbers yields a positive number.

The sum is positive and is therefore in true (uncomplemented) binary.

Positive number with magnitude larger than negative number:

$$\begin{array}{r} 00011111 \quad 15 \\ + 1111010 \quad + -6 \\ \hline 0001001 \quad 9 \end{array}$$

Discard carry $\longrightarrow$ 1

Addition of a positive number and a smaller negative number yields a positive number.

The final carry bit is discarded. The sum is positive and therefore in true (uncomplemented) binary.

Negative number with magnitude larger than positive number:

$$\begin{array}{r} 00010000 \quad 16 \\ + 11101000 \quad + -24 \\ \hline 11111000 \quad -8 \end{array}$$

Addition of a positive number and a larger negative number or two negative numbers yields a negative number in 2's complement.

The sum is negative and therefore in 2's complement form.

Both numbers negative:

$$\begin{array}{r} 11111011 \quad -5 \\ + 11110111 \quad + -9 \\ \hline 11110010 \quad -14 \end{array}$$

Discard carry $\longrightarrow$ 1

The final carry bit is discarded. The sum is negative and therefore in 2's complement form.

In a computer, the negative numbers are stored in 2's complement form so, as you can see, the addition process is very simple: *Add the two numbers and discard any final carry bit.*

Overflow Condition

When two numbers are added and the number of bits required to represent the sum exceeds the number of bits in the two numbers, an **overflow** results as indicated by an incorrect sign bit. An overflow can occur only when both numbers are positive or both numbers are negative. If the sign bit of the result is different than the sign bit of the numbers that are added, overflow is indicated. The following 8-bit example will illustrate this condition.

0111101

+ 0011010

1011011

125

+ 58

183

Sign incorrect

Magnitude incorrect

In this example the sum of 183 requires eight magnitude bits. Since there are seven magnitude bits in the numbers (one bit is the sign), there is a carry into the sign bit which produces the overflow indication.

Numbers Added Two at a Time

Now let's look at the addition of a string of numbers, added two at a time. This can be accomplished by adding the first two numbers, then adding the third number to the sum of the first two, then adding the fourth number to this result, and so on. This is how computers add strings of numbers. The addition of numbers taken two at a time is illustrated in Example 2–19.

EXAMPLE 2-19

Add the signed numbers: 01000100, 00011011, 00001110, and 00010010.

Solution

The equivalent decimal additions are given for reference.

68

+ 27

95

+ 14

109

+ 18

127

01000100

+ 00011011

01011111

+ 00001110

01101101

+ 00010010

01111111

Add 1st two numbers

1st sum

Add 3rd number

2nd sum

Add 4th number

Final sum

Related Problem

Add 00110011, 10111111, and 01100011. These are signed numbers.

Subtraction

Subtraction is addition with the sign of the subtrahend changed.

Subtraction is a special case of addition. For example, subtracting +6 (the **subtrahend**) from +9 (the **minuend**) is equivalent to adding -6 to +9. Basically, *the subtraction operation changes the sign of the subtrahend and adds it to the minuend.* The result of a subtraction is called the **difference**.

The sign of a positive or negative binary number is changed by taking its 2's complement.

For example, when you take the 2's complement of the positive number 00000100 (+4), you get 11111100, which is -4 as the following sum-of-weights evaluation shows:

$$-128 + 64 + 32 + 16 + 8 + 4 = -4$$

As another example, when you take the 2's complement of the negative number 11101101 (-19), you get 00010011, which is +19 as the following sum-of-weights evaluation shows:

$$16 + 2 + 1 = 19$$

Since subtraction is simply an addition with the sign of the subtrahend changed, the process is stated as follows:

To subtract two signed numbers, take the 2's complement of the subtrahend and add. Discard any final carry bit.

Example 2-20 illustrates the subtraction process.

When you subtract two binary numbers with the 2's complement method, it is important that both numbers have the same number of bits.

EXAMPLE 2-20

Perform each of the following subtractions of the signed numbers:

- (a) 00001000 - 00000011 (b) 00001100 - 11110111
(c) 11100111 - 00010011 (d) 10001000 - 11100010

Solution

Like in other examples, the equivalent decimal subtractions are given for reference.

- (a) In this case, $8 - 3 = 8 + (-3) = 5$.

00001000	Minuend (+8)
+ 11111101	2's complement of subtrahend (-3)
Discard carry → 1 0000101	Difference (+5)

- (b) In this case, $12 - (-9) = 12 + 9 = 21$.

00001100	Minuend (+12)
+ 00001001	2's complement of subtrahend (+9)
00010101	Difference (+21)

- (c) In this case, $-25 - (+19) = -25 + (-19) = -44$.

11100111	Minuend (-25)
+ 11101101	2's complement of subtrahend (-19)
Discard carry 1 11010100	Difference (-44)

- (d) In this case, $-120 - (-30) = -120 + 30 = -90$.

10001000	Minuend (-120)
+ 00011110	2's complement of subtrahend (+30)
10100110	Difference (-90)

Related Problem

Subtract 01000111 from 01011000.

Multiplication

The numbers in a multiplication are the **multiplicand**, the **multiplier**, and the **product**. These are illustrated in the following decimal multiplication:

$$\begin{array}{r} 8 \quad \text{Multiplicand} \\ \times 3 \quad \text{Multiplier} \\ \hline 24 \quad \text{Product} \end{array}$$

Multiplication is equivalent to adding a number to itself a number of times equal to the multiplier.

The multiplication operation in most computers is accomplished using addition. As you have already seen, subtraction is done with an adder; now let's see how multiplication is done.

Direct addition and *partial products* are two basic methods for performing multiplication using addition. In the direct addition method, you add the multiplicand a number of times equal to the multiplier. In the previous decimal example (8×3), three multiplicands are added: $8 + 8 + 8 = 24$. The disadvantage of this approach is that it becomes very lengthy if the multiplier is a large number. For example, to multiply 350×75 , you must add 350 to itself 75 times. Incidentally, this is why the term *times* is used to mean multiply.

When two binary numbers are multiplied, both numbers must be in true (uncomplemented) form. The direct addition method is illustrated in Example 2-21 adding two binary numbers at a time.

EXAMPLE 2-21

Multiply the signed binary numbers: 01001101 (multiplicand) and 00000100 (multiplier) using the direct addition method.

Solution

Since both numbers are positive, they are in true form, and the product will be positive. The decimal value of the multiplier is 4, so the multiplicand is added to itself four times as follows:

$$\begin{array}{r} 01001101 \quad \text{1st time} \\ + 01001101 \quad \text{2nd time} \\ \hline 10011010 \quad \text{Partial sum} \\ + 01001101 \quad \text{3rd time} \\ \hline 11100111 \quad \text{Partial sum} \\ + 01001101 \quad \text{4th time} \\ \hline \mathbf{100110100} \quad \text{Product} \end{array}$$

Since the sign bit of the multiplicand is 0, it has no effect on the outcome. All of the bits in the product are magnitude bits.

Related Problem

Multiply 01100001 by 00000110 using the direct addition method.

The partial products method is perhaps the more common one because it reflects the way you multiply longhand. The multiplicand is multiplied by each multiplier digit beginning with the least significant digit. The result of the multiplication of the multiplicand by a multiplier digit is called a *partial product*. Each successive partial product is moved (shifted) one place to the left and when all the partial products have been produced, they are added to get the final product. Here is a decimal example.

$$\begin{array}{r} 239 \quad \text{Multiplicand} \\ \times 123 \quad \text{Multiplier} \\ \hline 717 \quad \text{1st partial product (3} \times 239) \\ 478 \quad \text{2nd partial product (2} \times 239) \\ + 239 \quad \text{3rd partial product (1} \times 239) \\ \hline 29,397 \quad \text{Final product} \end{array}$$

The sign of the product of a multiplication depends on the signs of the multiplicand and the multiplier according to the following two rules:

- **If the signs are the same, the product is positive.**
- **If the signs are different, the product is negative.**

The basic steps in the partial products method of binary multiplication are as follows:

- Step 1:** Determine if the signs of the multiplicand and multiplier are the same or different. This determines what the sign of the product will be.
- Step 2:** Change any negative number to true (uncomplemented) form. Because most computers store negative numbers in 2's complement, a 2's complement operation is required to get the negative number into true form.
- Step 3:** Starting with the least significant multiplier bit, generate the partial products. When the multiplier bit is 1, the partial product is the same as the multiplicand. When the multiplier bit is 0, the partial product is zero. Shift each successive partial product one bit to the left.
- Step 4:** Add each successive partial product to the sum of the previous partial products to get the final product.
- Step 5:** If the sign bit that was determined in step 1 is negative, take the 2's complement of the product. If positive, leave the product in true form. Attach the sign bit to the product.

EXAMPLE 2-22

Multiply the signed binary numbers: 01010011 (multiplicand) and 11000101 (multiplier).

Solution

Step 1: The sign bit of the multiplicand is 0 and the sign bit of the multiplier is 1. The sign bit of the product will be 1 (negative).

Step 2: Take the 2's complement of the multiplier to put it in true form.

$$11000101 \longrightarrow 00111011$$

Step 3 and 4: The multiplication proceeds as follows. Notice that only the magnitude bits are used in these steps.

1010011	Multiplicand
$\times 0111011$	Multiplier
<u>1010011</u>	1st partial product
$+ 1010011$	2nd partial product
<u>11111001</u>	Sum of 1st and 2nd
$+ 0000000$	3rd partial product
<u>011111001</u>	Sum
$+ 1010011$	4th partial product
<u>1110010001</u>	Sum
$+ 1010011$	5th partial product
<u>100011000001</u>	Sum
$+ 1010011$	6th partial product
<u>1001100100001</u>	Sum
$+ 0000000$	7th partial product
<u>1001100100001</u>	Final product

Step 5: Since the sign of the product is a 1 as determined in step 1, take the 2's complement of the product.

1001100100001 $\longrightarrow$ 011001101111

Attach the sign bit $\swarrow$

1 011001101111

Related Problem

Verify the multiplication is correct by converting to decimal numbers and performing the multiplication.

Division

The numbers in a division are the **dividend**, the **divisor**, and the **quotient**. These are illustrated in the following standard division format.

$$\frac{\text{dividend}}{\text{divisor}} = \text{quotient}$$

The division operation in computers is accomplished using subtraction. Since subtraction is done with an adder, division can also be accomplished with an adder.

The result of a division is called the *quotient*; the quotient is the number of times that the divisor will go into the dividend. This means that the divisor can be subtracted from the dividend a number of times equal to the quotient, as illustrated by dividing 21 by 7.

21	Dividend
– 7	1st subtraction of divisor
14	1st partial remainder
– 7	2nd subtraction of divisor
7	2nd partial remainder
– 7	3rd subtraction of divisor
0	Zero remainder

In this simple example, the divisor was subtracted from the dividend three times before a remainder of zero was obtained. Therefore, the quotient is 3.

The sign of the quotient depends on the signs of the dividend and the divisor according to the following two rules:

- **If the signs are the same, the quotient is positive.**
- **If the signs are different, the quotient is negative.**

When two binary numbers are divided, both numbers must be in true (uncomplemented) form. The basic steps in a division process are as follows:

- Step 1:** Determine if the signs of the dividend and divisor are the same or different. This determines what the sign of the quotient will be. Initialize the quotient to zero.
- Step 2:** Subtract the divisor from the dividend using 2's complement addition to get the first partial remainder and add 1 to the quotient. If this partial remainder is positive, go to step 3. If the partial remainder is zero or negative, the division is complete.
- Step 3:** Subtract the divisor from the partial remainder and add 1 to the quotient. If the result is positive, repeat for the next partial remainder. If the result is zero or negative, the division is complete.

Continue to subtract the divisor from the dividend and the partial remainders until there is a zero or a negative result. Count the number of times that the divisor is subtracted and you have the quotient. Example 2–23 illustrates these steps using 8-bit signed binary numbers.

EXAMPLE 2-23

Divide 01100100 by 00011001.

Solution

Step 1: The signs of both numbers are positive, so the quotient will be positive. The quotient is initially zero: 00000000.

Step 2: Subtract the divisor from the dividend using 2's complement addition (remember that final carries are discarded).

01100100	Dividend
+ 11100111	2's complement of divisor
01001011	Positive 1st partial remainder

Add 1 to quotient: 00000000 + 00000001 = 00000001.

Step 3: Subtract the divisor from the 1st partial remainder using 2's complement addition.

01001011	1st partial remainder
+ 11100111	2's complement of divisor
00110010	Positive 2nd partial remainder

Add 1 to quotient: 00000001 + 00000001 = 00000010.

Step 4: Subtract the divisor from the 2nd partial remainder using 2's complement addition.

00110010	2nd partial remainder
+ 11100111	2's complement of divisor
00011001	Positive 3rd partial remainder

Add 1 to quotient: 00000010 + 00000001 = 00000011.

Step 5: Subtract the divisor from the 3rd partial remainder using 2's complement addition.

00011001	3rd partial remainder
+ 11100111	2's complement of divisor
00000000	Zero remainder

Add 1 to quotient: 00000011 + 00000001 = **00000100** (final quotient). The process is complete.

Related Problem

Verify that the process is correct by converting to decimal numbers and performing the division.

SECTION 2-7 CHECKUP

1. List the four cases when numbers are added.
2. Add the signed numbers 00100001 and 10111100.
3. Subtract the signed numbers 00110010 from 01110111.
4. What is the sign of the product when two negative numbers are multiplied?
5. Multiply 01111111 by 00000101.
6. What is the sign of the quotient when a positive number is divided by a negative number?
7. Divide 00110000 by 00001100.

2-8 Hexadecimal Numbers

The hexadecimal number system has sixteen characters; it is used primarily as a compact way of displaying or writing binary numbers because it is very easy to convert between binary and hexadecimal. As you are probably aware, long binary numbers are difficult to read and write because it is easy to drop or transpose a bit. Since computers and microprocessors understand only 1s and 0s, it is necessary to use these digits when you program in “machine language.” Imagine writing a sixteen bit instruction for a microprocessor system in 1s and 0s. It is much more efficient to use hexadecimal or octal; octal numbers are covered in Section 2-9. Hexadecimal is widely used in computer and microprocessor applications.

After completing this section, you should be able to

- ◆ List the hexadecimal characters
- ◆ Count in hexadecimal
- ◆ Convert from binary to hexadecimal
- ◆ Convert from hexadecimal to binary
- ◆ Convert from hexadecimal to decimal
- ◆ Convert from decimal to hexadecimal
- ◆ Add hexadecimal numbers
- ◆ Determine the 2’s complement of a hexadecimal number
- ◆ Subtract hexadecimal numbers

The hexadecimal number system consists of digits 0–9 and letters A–F.

The **hexadecimal** number system has a base of sixteen; that is, it is composed of 16 **numeric** and alphabetic **characters**. Most digital systems process binary data in groups that are multiples of four bits, making the hexadecimal number very convenient because each hexadecimal digit represents a 4-bit binary number (as listed in Table 2-3).

TABLE 2-3		
Decimal	Binary	Hexadecimal
0	0000	0
1	0001	1
2	0010	2
3	0011	3
4	0100	4
5	0101	5
6	0110	6
7	0111	7
8	1000	8
9	1001	9
10	1010	A
11	1011	B
12	1100	C
13	1101	D
14	1110	E
15	1111	F

Ten numeric digits and six alphabetic characters make up the hexadecimal number system. The use of letters A, B, C, D, E, and F to represent numbers may seem strange at first, but keep in mind that any number system is only a set of sequential symbols. If you understand what quantities these symbols represent, then the form of the symbols

themselves is less important once you get accustomed to using them. We will use the subscript 16 to designate hexadecimal numbers to avoid confusion with decimal numbers. Sometimes you may see an “h” following a hexadecimal number.

Counting in Hexadecimal

How do you count in hexadecimal once you get to F? Simply start over with another column and continue as follows:

... , E, F, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 1A, 1B, 1C, 1D, 1E, 1F, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 2A, 2B, 2C, 2D, 2E, 2F, 30, 31, ...

With two hexadecimal digits, you can count up to FF_{16} , which is decimal 255. To count beyond this, three hexadecimal digits are needed. For instance, 100_{16} is decimal 256, 101_{16} is decimal 257, and so forth. The maximum 3-digit hexadecimal number is FFF_{16} , or decimal 4095. The maximum 4-digit hexadecimal number is $FFFF_{16}$, which is decimal 65,535.

Binary-to-Hexadecimal Conversion

Converting a binary number to hexadecimal is a straightforward procedure. Simply break the binary number into 4-bit groups, starting at the right-most bit and replace each 4-bit group with the equivalent hexadecimal symbol.

EXAMPLE 2-24

Convert the following binary numbers to hexadecimal:

(a) 1100101001010111 (b) 111111000101101001

Solution

(a) $\overbrace{1100}^C \overbrace{1010}^A \overbrace{0101}^5 \overbrace{0111}^7} = CA57_{16}$ (b) $\overbrace{0011}^3 \overbrace{1111}^F \overbrace{0001}^1 \overbrace{0110}^6 \overbrace{1001}^9} = 3F169_{16}$

Two zeros have been added in part (b) to complete a 4-bit group at the left.

Related Problem

Convert the binary number 1001111011110011100 to hexadecimal.

Hexadecimal-to-Binary Conversion

To convert from a hexadecimal number to a binary number, reverse the process and replace each hexadecimal symbol with the appropriate four bits.

EXAMPLE 2-25

Determine the binary numbers for the following hexadecimal numbers:

(a) $10A4_{16}$ (b) $CF8E_{16}$ (c) 9742_{16}

Solution

(a) $\overbrace{1000}^1 \overbrace{0101}^0 \overbrace{1001}^A \overbrace{0100}^4} = 1000010100100$ (b) $\overbrace{1100}^C \overbrace{1111}^F \overbrace{1000}^8 \overbrace{1110}^E} = 1100111110001110$ (c) $\overbrace{1001}^9 \overbrace{0111}^7 \overbrace{1010}^4 \overbrace{0001}^2} = 10010111101000010$

In part (a), the MSB is understood to have three zeros preceding it, thus forming a 4-bit group.

Related Problem

Convert the hexadecimal number 6BD3 to binary.

InfoNote

With memories in the gigabyte (GB) range, specifying a memory address in binary is quite cumbersome. For example, it takes 32 bits to specify an address in a 4 GB memory. It is much easier to express a 32-bit code using 8 hexadecimal digits.

Hexadecimal is a convenient way to represent binary numbers.

Conversion between hexadecimal and binary is direct and easy.

It should be clear that it is much easier to deal with a hexadecimal number than with the equivalent binary number. Since conversion is so easy, the hexadecimal system is widely used for representing binary numbers in programming, printouts, and displays.

Hexadecimal-to-Decimal Conversion

One way to find the decimal equivalent of a hexadecimal number is to first convert the hexadecimal number to binary and then convert from binary to decimal.

EXAMPLE 2-26

Convert the following hexadecimal numbers to decimal:

(a) 1C₁₆ (b) A85₁₆

Solution

Remember, convert the hexadecimal number to binary first, then to decimal.

(a)
$$\begin{array}{cc} 1 & C \\ \downarrow & \downarrow \\ 0001 & 1100 \end{array} = 2^4 + 2^3 + 2^2 = 16 + 8 + 4 = \mathbf{28}_{10}$$

(b)
$$\begin{array}{ccc} A & 8 & 5 \\ \downarrow & \downarrow & \downarrow \\ 1010 & 1000 & 101 \end{array} = 2^{11} + 2^9 + 2^7 + 2^2 + 2^0 = 2048 + 512 + 128 + 4 + 1 = \mathbf{2693}_{10}$$

Related Problem

Convert the hexadecimal number 6BD to decimal.

A calculator can be used to perform arithmetic operations with hexadecimal numbers.

Another way to convert a hexadecimal number to its decimal equivalent is to multiply the decimal value of each hexadecimal digit by its weight and then take the sum of these products. The weights of a hexadecimal number are increasing powers of 16 (from right to left). For a 4-digit hexadecimal number, the weights are

16 ³	16 ²	16 ¹	16 ⁰
4096	256	16	1

CALCULATOR SESSION

Conversion of a Hexadecimal Number to a Decimal Number

Convert hexadecimal 28A to decimal.

TI-36X Step 1: 3rd (HEX

Step 2: 2 8 3rd 1/x A

Step 3: 3rd EE DEC

650

EXAMPLE 2-27

Convert the following hexadecimal numbers to decimal:

(a) E5₁₆ (b) B2F8₁₆

Solution

Recall from Table 2-3 that letters A through F represent decimal numbers 10 through 15, respectively.

(a) $E5_{16} = (E \times 16) + (5 \times 1) = (14 \times 16) + (5 \times 1) = 224 + 5 = \mathbf{229}_{10}$

(b) $B2F8_{16} = (B \times 4096) + (2 \times 256) + (F \times 16) + (8 \times 1)$
 $= (11 \times 4096) + (2 \times 256) + (15 \times 16) + (8 \times 1)$
 $= \mathbf{45,056} + \mathbf{512} + \mathbf{240} + \mathbf{8} = \mathbf{45,816}_{10}$

Related Problem

Convert 60A₁₆ to decimal.

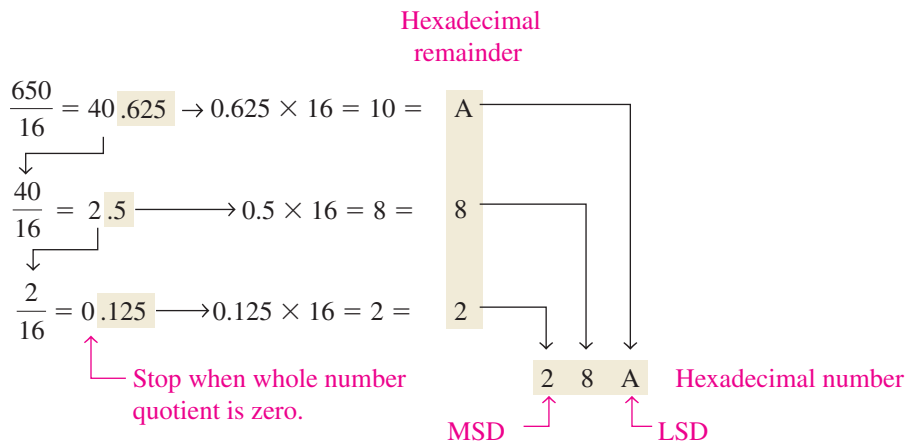
Decimal-to-Hexadecimal Conversion

Repeated division of a decimal number by 16 will produce the equivalent hexadecimal number, formed by the remainders of the divisions. The first remainder produced is the least significant digit (LSD). Each successive division by 16 yields a remainder that becomes a digit in the equivalent hexadecimal number. This procedure is similar to repeated division by 2 for decimal-to-binary conversion that was covered in Section 2–3. Example 2–28 illustrates the procedure. Note that when a quotient has a fractional part, the fractional part is multiplied by the divisor to get the remainder.

EXAMPLE 2-28

Convert the decimal number 650 to hexadecimal by repeated division by 16.

Solution



Related Problem

Convert decimal 2591 to hexadecimal.

Hexadecimal Addition

Addition can be done directly with hexadecimal numbers by remembering that the hexadecimal digits 0 through 9 are equivalent to decimal digits 0 through 9 and that hexadecimal digits A through F are equivalent to decimal numbers 10 through 15. When adding two hexadecimal numbers, use the following rules. (Decimal numbers are indicated by a subscript 10.)

1. In any given column of an addition problem, think of the two hexadecimal digits in terms of their decimal values. For instance, $5_{16} = 5_{10}$ and $C_{16} = 12_{10}$.
2. If the sum of these two digits is 15_{10} or less, bring down the corresponding hexadecimal digit.
3. If the sum of these two digits is greater than 15_{10} , bring down the amount of the sum that exceeds 16_{10} and carry a 1 to the next column.

EXAMPLE 2-29

Add the following hexadecimal numbers:

- (a) $23_{16} + 16_{16}$ (b) $58_{16} + 22_{16}$ (c) $2B_{16} + 84_{16}$ (d) $DF_{16} + AC_{16}$

Solution

(a)
$$\begin{array}{r} 23_{16} \\ + 16_{16} \\ \hline 39_{16} \end{array}$$

right column: $3_{16} + 6_{16} = 3_{10} + 6_{10} = 9_{10} = 9_{16}$
left column: $2_{16} + 1_{16} = 2_{10} + 1_{10} = 3_{10} = 3_{16}$

CALCULATOR SESSION

Conversion of a Decimal Number to a Hexadecimal Number

Convert decimal 650 to hexadecimal.

TI-36X DEC Step 1: 3rd EE Step 2: 6 5 0 HEX Step 3: 3rd (28A

(b)

58₁₆

+ 22₁₆

7A₁₆

right column:

8₁₆ + 2₁₆ = 8₁₀ + 2₁₀ = 10₁₀ = A₁₆

left column:

5₁₆ + 2₁₆ = 5₁₀ + 2₁₀ = 7₁₀ = 7₁₆

(c)

2B₁₆

+ 84₁₆

AF₁₆

right column:

B₁₆ + 4₁₆ = 11₁₀ + 4₁₀ = 15₁₀ = F₁₆

left column:

2₁₆ + 8₁₆ = 2₁₀ + 8₁₀ = 10₁₀ = A₁₆

(d)

DF₁₆

+ AC₁₆

18B₁₆

right column:

F₁₆ + C₁₆ = 15₁₀ + 12₁₀ = 27₁₀

27₁₀ - 16₁₀ = 11₁₀ = B₁₆ with a 1 carry

left column:

D₁₆ + A₁₆ + 1₁₆ = 13₁₀ + 10₁₀ + 1₁₀ = 24₁₀

24₁₀ - 16₁₀ = 8₁₀ = 8₁₆ with a 1 carry

Related Problem

Add 4C₁₆ and 3A₁₆.

Hexadecimal Subtraction

As you have learned, the 2’s complement allows you to subtract by adding binary numbers. Since a hexadecimal number can be used to represent a binary number, it can also be used to represent the 2’s complement of a binary number.

There are three ways to get the 2’s complement of a hexadecimal number. Method 1 is the most common and easiest to use. Methods 2 and 3 are alternate methods.

Method 1: Convert the hexadecimal number to binary. Take the 2’s complement of the binary number. Convert the result to hexadecimal. This is illustrated in Figure 2–4.

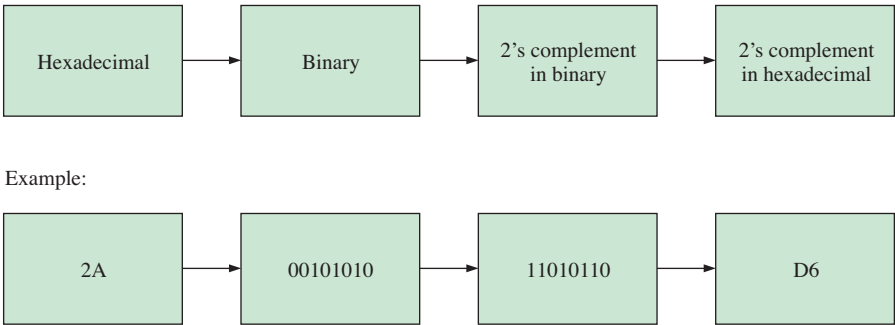


FIGURE 2–4 Getting the 2’s complement of a hexadecimal number, Method 1.

Method 2: Subtract the hexadecimal number from the maximum hexadecimal number and add 1. This is illustrated in Figure 2–5.

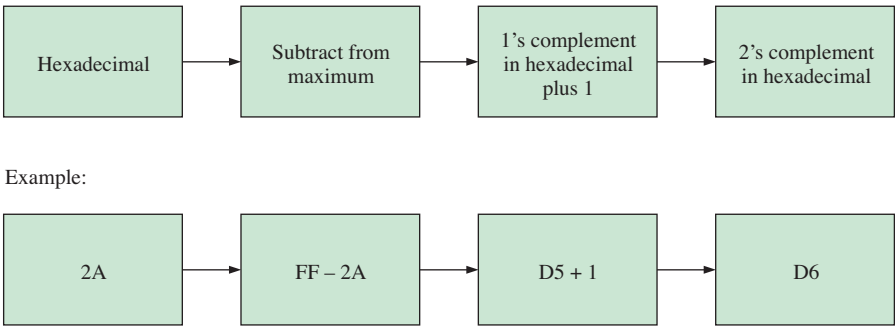


FIGURE 2–5 Getting the 2’s complement of a hexadecimal number, Method 2.

Method 3: Write the sequence of single hexadecimal digits. Write the sequence in reverse below the forward sequence. The 1's complement of each hex digit is the digit directly below it. Add 1 to the resulting number to get the 2's complement. This is illustrated in Figure 2–6.

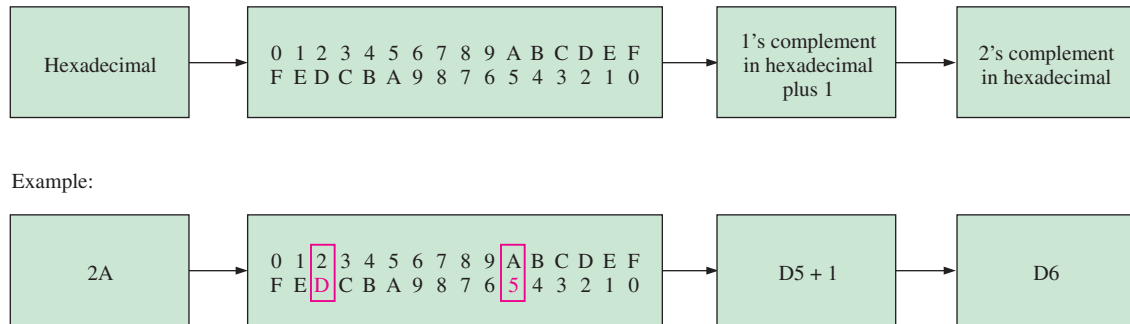


FIGURE 2-6 Getting the 2's complement of a hexadecimal number, Method 3.

EXAMPLE 2-30

Subtract the following hexadecimal numbers:

- (a) $84_{16} - 2A_{16}$ (b) $C3_{16} - 0B_{16}$

Solution

- (a) $2A_{16} = 00101010$

2's complement of $2A_{16} = 11010110 = D6_{16}$ (using Method 1)

$$\begin{array}{r} 84_{16} \\ + D6_{16} \\ \hline 15A_{16} \end{array} \quad \begin{array}{l} \text{Add} \\ \text{Drop carry, as in 2's complement addition} \end{array}$$

The difference is $5A_{16}$.

- (b) $0B_{16} = 00001011$

2's complement of $0B_{16} = 11110101 = F5_{16}$ (using Method 1)

$$\begin{array}{r} C3_{16} \\ + F5_{16} \\ \hline 1B8_{16} \end{array} \quad \begin{array}{l} \text{Add} \\ \text{Drop carry} \end{array}$$

The difference is $B8_{16}$.

Related Problem

Subtract 173_{16} from BCD_{16} .

SECTION 2-8 CHECKUP

- Convert the following binary numbers to hexadecimal:
 - 10110011
 - 110011101000
- Convert the following hexadecimal numbers to binary:
 - 57_{16}
 - $3A5_{16}$
 - $F80B_{16}$
- Convert $9B30_{16}$ to decimal.
- Convert the decimal number 573 to hexadecimal.

5. Add the following hexadecimal numbers directly:

(a) $18_{16} + 34_{16}$ (b) $3F_{16} + 2A_{16}$

6. Subtract the following hexadecimal numbers:

(a) $75_{16} - 21_{16}$ (b) $94_{16} - 5C_{16}$

2-9 Octal Numbers

Like the hexadecimal number system, the octal number system provides a convenient way to express binary numbers and codes. However, it is used less frequently than hexadecimal in conjunction with computers and microprocessors to express binary quantities for input and output purposes.

After completing this section, you should be able to

- ♦ Write the digits of the octal number system
- ♦ Convert from octal to decimal
- ♦ Convert from decimal to octal
- ♦ Convert from octal to binary
- ♦ Convert from binary to octal

The **octal** number system is composed of eight digits, which are

0, 1, 2, 3, 4, 5, 6, 7

To count above 7, begin another column and start over:

10, 11, 12, 13, 14, 15, 16, 17, 20, 21, . . .

The octal number system has a base of 8.

Counting in octal is similar to counting in decimal, except that the digits 8 and 9 are not used. To distinguish octal numbers from decimal numbers or hexadecimal numbers, we will use the subscript 8 to indicate an octal number. For instance, 15_8 in octal is equivalent to 13_{10} in decimal and D in hexadecimal. Sometimes you may see an “o” or a “Q” following an octal number.

Octal-to-Decimal Conversion

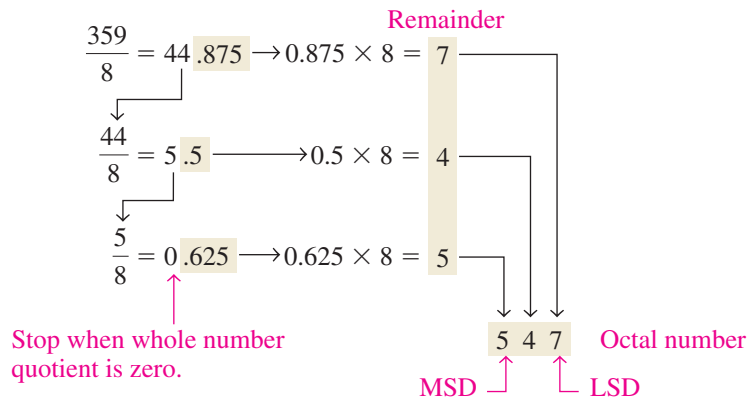
Since the octal number system has a base of eight, each successive digit position is an increasing power of eight, beginning in the right-most column with 8^0 . The evaluation of an octal number in terms of its decimal equivalent is accomplished by multiplying each digit by its weight and summing the products, as illustrated here for 2374_8 .

$$\begin{array}{r}
 \text{Weight: } 8^3 \ 8^2 \ 8^1 \ 8^0 \\
 \text{Octal number: } 2 \ 3 \ 7 \ 4 \\
 2374_8 = (2 \times 8^3) + (3 \times 8^2) + (7 \times 8^1) + (4 \times 8^0) \\
 = (2 \times 512) + (3 \times 64) + (7 \times 8) + (4 \times 1) \\
 = 1024 + 192 + 56 + 4 = 1276_{10}
 \end{array}$$

Decimal-to-Octal Conversion

A method of converting a decimal number to an octal number is the repeated division-by-8 method, which is similar to the method used in the conversion of decimal numbers to binary or to hexadecimal. To show how it works, let's convert the decimal number 359 to

octal. Each successive division by 8 yields a remainder that becomes a digit in the equivalent octal number. The first remainder generated is the least significant digit (LSD).



CALCULATOR SESSION

Conversion of a Decimal Number to an Octal Number

Convert decimal 439 to octal.

TI-36X DEC

Step 1: 3rd EE

Step 2: 4 3 9

OCT

Step 3: 3rd)

667

Octal-to-Binary Conversion

Because each octal digit can be represented by a 3-bit binary number, it is very easy to convert from octal to binary. Each octal digit is represented by three bits as shown in Table 2-4.

Octal is a convenient way to represent binary numbers, but it is not as commonly used as hexadecimal.

TABLE 2-4

Octal/binary conversion.

Octal Digit	0	1	2	3	4	5	6	7
Binary	000	001	010	011	100	101	110	111

To convert an octal number to a binary number, simply replace each octal digit with the appropriate three bits.

EXAMPLE 2-31

Convert each of the following octal numbers to binary:

- (a) 13_8 (b) 25_8 (c) 140_8 (d) 7526_8

Solution

- (a) $\begin{array}{cc} 1 & 3 \\ \downarrow & \downarrow \\ 00 & 1011 \end{array}$ (b) $\begin{array}{cc} 2 & 5 \\ \downarrow & \downarrow \\ 010 & 101 \end{array}$ (c) $\begin{array}{ccc} 1 & 4 & 0 \\ \downarrow & \downarrow & \downarrow \\ 001 & 100 & 000 \end{array}$ (d) $\begin{array}{cccc} 7 & 5 & 2 & 6 \\ \downarrow & \downarrow & \downarrow & \downarrow \\ 111 & 101 & 010 & 110 \end{array}$

Related Problem

Convert each of the binary numbers to decimal and verify that each value agrees with the decimal value of the corresponding octal number.

Binary-to-Octal Conversion

Conversion of a binary number to an octal number is the reverse of the octal-to-binary conversion. The procedure is as follows: Start with the right-most group of three bits and, moving from right to left, convert each 3-bit group to the equivalent octal digit. If there are not three bits available for the left-most group, add either one or two zeros to make a complete group. These leading zeros do not affect the value of the binary number.

EXAMPLE 2-32

Convert each of the following binary numbers to octal:

- (a) 110101 (b) 101111001 (c) 100110011010 (d) 11010000100

Solution

$$\begin{array}{c} \text{(a)} \quad 110101 \\ \downarrow \quad \downarrow \\ 6 \quad 5 = 65_8 \end{array}$$

$$\begin{array}{c} \text{(b)} \quad 101111001 \\ \downarrow \quad \downarrow \quad \downarrow \\ 5 \quad 7 \quad 1 = 571_8 \end{array}$$

$$\begin{array}{c} \text{(c)} \quad 100110011010 \\ \downarrow \quad \downarrow \quad \downarrow \quad \downarrow \\ 4 \quad 6 \quad 3 \quad 2 = 4632_8 \end{array}$$

$$\begin{array}{c} \text{(d)} \quad 011010000100 \\ \downarrow \quad \downarrow \quad \downarrow \quad \downarrow \\ 3 \quad 2 \quad 0 \quad 4 = 3204_8 \end{array}$$

Related Problem

Convert the binary number 1010101000111110010 to octal.

SECTION 2-9 CHECKUP

1. Convert the following octal numbers to decimal:

- (a) 73_8 (b) 125_8

2. Convert the following decimal numbers to octal:

- (a) 98_{10} (b) 163_{10}

3. Convert the following octal numbers to binary:

- (a) 46_8 (b) 723_8 (c) 5624_8

4. Convert the following binary numbers to octal:

- (a) 110101111 (b) 1001100010 (c) 10111111001

2-10 Binary Coded Decimal (BCD)

Binary coded decimal (BCD) is a way to express each of the decimal digits with a binary code. There are only ten code groups in the BCD system, so it is very easy to convert between decimal and BCD. Because we like to read and write in decimal, the BCD code provides an excellent interface to binary systems. Examples of such interfaces are keypad inputs and digital readouts.

After completing this section, you should be able to

- ♦ Convert each decimal digit to BCD
- ♦ Express decimal numbers in BCD
- ♦ Convert from BCD to decimal
- ♦ Add BCD numbers

The 8421 BCD Code

The 8421 code is a type of **BCD** (binary coded decimal) code. Binary coded decimal means that each decimal digit, 0 through 9, is represented by a binary code of four bits. The designation 8421 indicates the binary weights of the four bits (2^3 , 2^2 , 2^1 , 2^0). The ease of conversion between 8421 code numbers and the familiar decimal numbers is the main advantage

In BCD, 4 bits represent each decimal digit.

of this code. All you have to remember are the ten binary combinations that represent the ten decimal digits as shown in Table 2–5. The 8421 code is the predominant BCD code, and when we refer to BCD, we always mean the 8421 code unless otherwise stated.

TABLE 2–5

Decimal/BCD conversion.

Decimal Digit	0	1	2	3	4	5	6	7	8	9
BCD	0000	0001	0010	0011	0100	0101	0110	0111	1000	1001

Invalid Codes

You should realize that, with four bits, sixteen numbers (0000 through 1111) can be represented but that, in the 8421 code, only ten of these are used. The six code combinations that are not used—1010, 1011, 1100, 1101, 1110, and 1111—are invalid in the 8421 BCD code.

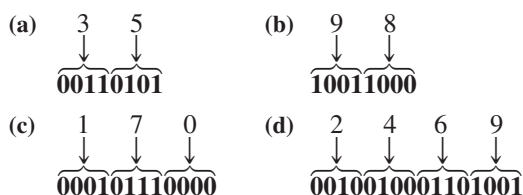
To express any decimal number in BCD, simply replace each decimal digit with the appropriate 4-bit code, as shown by Example 2–33.

EXAMPLE 2–33

Convert each of the following decimal numbers to BCD:

- (a) 35 (b) 98 (c) 170 (d) 2469

Solution



Related Problem

Convert the decimal number 9673 to BCD.

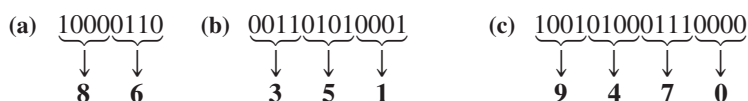
It is equally easy to determine a decimal number from a BCD number. Start at the right-most bit and break the code into groups of four bits. Then write the decimal digit represented by each 4-bit group.

EXAMPLE 2–34

Convert each of the following BCD codes to decimal:

- (a) 10000110 (b) 001101010001 (c) 1001010001110000

Solution



Related Problem

Convert the BCD code 10000010001001110110 to decimal.

InfoNote

BCD is sometimes used for arithmetic operations in processors. To represent BCD numbers in a processor, they usually are “packed,” so that eight bits have two BCD digits. Normally, a processor will add numbers as if they were straight binary. Special instructions are available for computer programmers to correct the results when BCD numbers are added or subtracted. For example, in Assembly Language, the programmer will include a DAA (Decimal Adjust for Addition) instruction to automatically correct the answer to BCD following an addition.

Applications

Digital clocks, digital thermometers, digital meters, and other devices with seven-segment displays typically use BCD code to simplify the displaying of decimal numbers. BCD is not as efficient as straight binary for calculations, but it is particularly useful if only limited processing is required, such as in a digital thermometer.

BCD Addition

BCD is a numerical code and can be used in arithmetic operations. Addition is the most important operation because the other three operations (subtraction, multiplication, and division) can be accomplished by the use of addition. Here is how to add two BCD numbers:

- Step 1:** Add the two BCD numbers, using the rules for binary addition in Section 2–4.
- Step 2:** If a 4-bit sum is equal to or less than 9, it is a valid BCD number.
- Step 3:** If a 4-bit sum is greater than 9, or if a carry out of the 4-bit group is generated, it is an invalid result. Add 6 (0110) to the 4-bit sum in order to skip the six invalid states and return the code to 8421. If a carry results when 6 is added, simply add the carry to the next 4-bit group.

Example 2–35 illustrates BCD additions in which the sum in each 4-bit column is equal to or less than 9, and the 4-bit sums are therefore valid BCD numbers. Example 2–36 illustrates the procedure in the case of invalid sums (greater than 9 or a carry).

An alternative method to add BCD numbers is to convert them to decimal, perform the addition, and then convert the answer back to BCD.

EXAMPLE 2-35

Add the following BCD numbers:

- (a) 0011 + 0100 (b) 00100011 + 00010101
(c) 10000110 + 00010011 (d) 010001010000 + 010000010111

Solution

The decimal number additions are shown for comparison.

(a)	0011	3	(b)	0010	0011	23		
	+ 0100	+ 4		+ 0001	0101	+ 15		
	0111	7		0011	1000	38		
(c)	1000	0110	86	(d)	0100	0101	0000	450
	+ 0001	0011	+ 13		+ 0100	0001	0111	+ 417
	1001	1001	99		1000	0110	0111	867

Note that in each case the sum in any 4-bit column does not exceed 9, and the results are valid BCD numbers.

Related Problem

Add the BCD numbers: 1001000001000011 + 0000100100100101.

EXAMPLE 2-36

Add the following BCD numbers:

- (a) 1001 + 0100 (b) 1001 + 1001
(c) 00010110 + 00010101 (d) 01100111 + 01010011

Solution

The decimal number additions are shown for comparison.

(a)

1001		9
+ 0100		+ 4
1101	Invalid BCD number (>9)	13
+ 0110	Add 6	
<u>0001</u> <u>0011</u>	Valid BCD number	
↓ ↓		
1 3		

(b)

1001		9
+ 1001		+ 9
1 0010	Invalid because of carry	18
+ 0110	Add 6	
<u>0001</u> <u>1000</u>	Valid BCD number	
↓ ↓		
1 8		

(c)

0001	0110		16
+ 0001	0101		+ 15
0010	1011	Right group is invalid (>9), left group is valid.	31
	+ 0110	Add 6 to invalid code. Add carry, 0001, to next group.	
<u>0011</u> <u>0001</u>	Valid BCD number		
↓ ↓			
3 1			

(d)

0110	0111		67
+ 0101	0011		+ 53
1011	1010	Both groups are invalid (>9)	120
+ 0110	+ 0110	Add 6 to both groups	
<u>0001</u> <u>0010</u> <u>0000</u>	Valid BCD number		
↓ ↓ ↓			
1 2 0			

Related Problem

Add the BCD numbers: 01001000 + 00110100.

SECTION 2-10 CHECKUP

- What is the binary weight of each 1 in the following BCD numbers?
 (a) 0010 (b) 1000 (c) 0001 (d) 0100
- Convert the following decimal numbers to BCD:
 (a) 6 (b) 15 (c) 273 (d) 849
- What decimal numbers are represented by each BCD code?
 (a) 10001001 (b) 001001111000 (c) 000101010111
- In BCD addition, when is a 4-bit sum invalid?

2-11 Digital Codes

Many specialized codes are used in digital systems. You have just learned about the BCD code; now let's look at a few others. Some codes are strictly numeric, like BCD, and others are alphanumeric; that is, they are used to represent numbers, letters, symbols, and instructions. The codes introduced in this section are the Gray code, the ASCII code, and the Unicode.

After completing this section, you should be able to

- ◆ Explain the advantage of the Gray code
- ◆ Convert between Gray code and binary
- ◆ Use the ASCII code
- ◆ Discuss the Unicode

The Gray Code

The single bit change characteristic of the Gray code minimizes the chance for error.

The **Gray code** is unweighted and is not an arithmetic code; that is, there are no specific weights assigned to the bit positions. The important feature of the Gray code is that *it exhibits only a single bit change from one code word to the next in sequence*. This property is important in many applications, such as shaft position encoders, where error susceptibility increases with the number of bit changes between adjacent numbers in a sequence.

Table 2-6 is a listing of the 4-bit Gray code for decimal numbers 0 through 15. Binary numbers are shown in the table for reference. Like binary numbers, *the Gray code can have any number of bits*. Notice the single-bit change between successive Gray code words. For instance, in going from decimal 3 to decimal 4, the Gray code changes from 0010 to 0110, while the binary code changes from 0011 to 0100, a change of three bits. The only bit change in the Gray code is in the third bit from the right: the other bits remain the same.

TABLE 2-6
Four-bit Gray code.

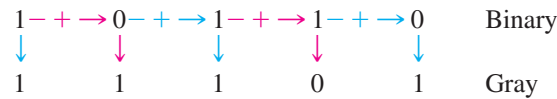
Decimal	Binary	Gray Code	Decimal	Binary	Gray Code
0	0000	0000	8	1000	1100
1	0001	0001	9	1001	1101
2	0010	0011	10	1010	1111
3	0011	0010	11	1011	1110
4	0100	0110	12	1100	1010
5	0101	0111	13	1101	1011
6	0110	0101	14	1110	1001
7	0111	0100	15	1111	1000

Binary-to-Gray Code Conversion

Conversion between binary code and Gray code is sometimes useful. The following rules explain how to convert from a binary number to a Gray code word:

1. The most significant bit (left-most) in the Gray code is the same as the corresponding MSB in the binary number.
2. Going from left to right, add each adjacent pair of binary code bits to get the next Gray code bit. Discard carries.

For example, the conversion of the binary number 10110 to Gray code is as follows:



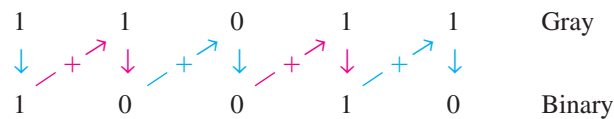
The Gray code is 11101.

Gray-to-Binary Code Conversion

To convert from Gray code to binary, use a similar method; however, there are some differences. The following rules apply:

1. The most significant bit (left-most) in the binary code is the same as the corresponding bit in the Gray code.
2. Add each binary code bit generated to the Gray code bit in the next adjacent position. Discard carries.

For example, the conversion of the Gray code word 11011 to binary is as follows:



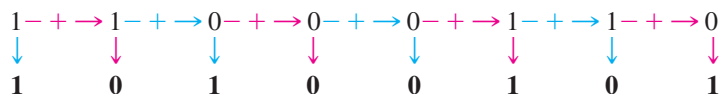
The binary number is 10010.

EXAMPLE 2-37

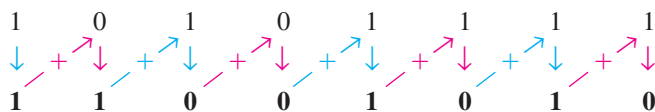
- (a) Convert the binary number 11000110 to Gray code.
- (b) Convert the Gray code 10101111 to binary.

Solution

- (a) Binary to Gray code:



- (b) Gray code to binary:



Related Problem

- (a) Convert binary 101101 to Gray code.
- (b) Convert Gray code 100111 to binary.

An Application

The concept of a 3-bit shaft position encoder is shown in Figure 2-7. Basically, there are three concentric rings that are segmented into eight sectors. The more sectors there are, the more accurately the position can be represented, but we are using only eight to illustrate. Each sector of each ring is either reflective or nonreflective. As the rings rotate with the shaft, they come under an IR emitter that produces three separate IR beams. A 1 is indicated where there is a reflected beam, and a 0 is indicated where there is no reflected beam. The IR detector senses the presence or absence of reflected

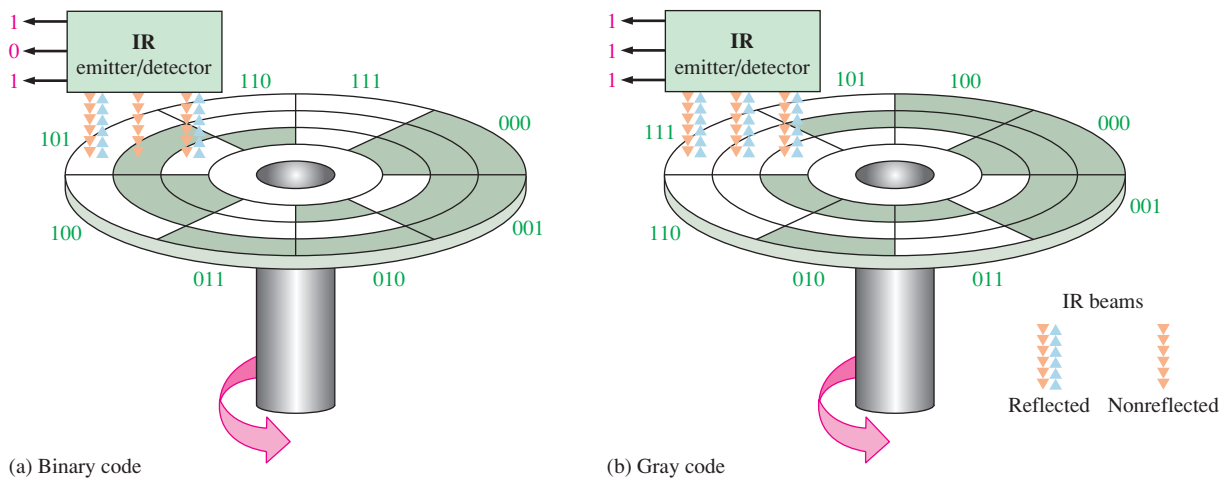


FIGURE 2-7 A simplified illustration of how the Gray code solves the error problem in shaft position encoders. Three bits are shown to illustrate the concept, although most shaft encoders use more than 10 bits to achieve a higher resolution.

beams and produces a corresponding 3-bit code. The IR emitter/detector is in a fixed position. As the shaft rotates counterclockwise through 360° , the eight sectors move under the three beams. Each beam is either reflected or absorbed by the sector surface to represent a binary or Gray code number that indicates the shaft position.

In Figure 2-7(a), the sectors are arranged in a straight binary pattern, so that the detector output goes from 000 to 001 to 010 to 011 and so on. When a beam is aligned over a reflective sector, the output is 1; when a beam is aligned over a nonreflective sector, the output is 0. If one beam is slightly ahead of the others during the transition from one sector to the next, an erroneous output can occur. Consider what happens when the beams are on the 111 sector and about to enter the 000 sector. If the MSB beam is slightly ahead, the position would be incorrectly indicated by a transitional 011 instead of a 111 or a 000. In this type of application, it is virtually impossible to maintain precise mechanical alignment of the IR emitter/detector beams; therefore, some error will usually occur at many of the transitions between sectors.

The Gray code is used to eliminate the error problem which is inherent in the binary code. As shown in Figure 2-7(b), the Gray code assures that only one bit will change between adjacent sectors. This means that even though the beams may not be in precise alignment, there will never be a transitional error. For example, let's again consider what happens when the beams are on the 111 sector and about to move into the next sector, 101. The only two possible outputs during the transition are 111 and 101, no matter how the beams are aligned. A similar situation occurs at the transitions between each of the other sectors.

Alphanumeric Codes

In order to communicate, you need not only numbers, but also letters and other symbols. In the strictest sense, **alphanumeric** codes are codes that represent numbers and alphabetic characters (letters). Most such codes, however, also represent other characters such as symbols and various instructions necessary for conveying information.

At a minimum, an alphanumeric code must represent 10 decimal digits and 26 letters of the alphabet, for a total of 36 items. This number requires six bits in each code combination because five bits are insufficient ($2^5 = 32$). There are 64 total combinations of six bits, so there are 28 unused code combinations. Obviously, in many applications, symbols other than just numbers and letters are necessary to communicate completely. You need spaces, periods, colons, semicolons, question marks, etc. You also need instructions to tell the receiving system what to do with the information. With codes that are six bits long, you can handle decimal numbers, the alphabet, and 28 other symbols. This should give you an idea of the requirements for a basic alphanumeric code. The ASCII is a common alphanumeric code and is covered next.

ASCII

ASCII is the abbreviation for American Standard Code for Information Interchange. Pronounced “askee,” ASCII is a universally accepted alphanumeric code used in most computers and other electronic equipment. Most computer keyboards are standardized with the ASCII. When you enter a letter, a number, or control command, the corresponding ASCII code goes into the computer.

ASCII has 128 characters and symbols represented by a 7-bit binary code. Actually, ASCII can be considered an 8-bit code with the MSB always 0. This 8-bit code is 00 through 7F in hexadecimal. The first thirty-two ASCII characters are nongraphic commands that are never printed or displayed and are used only for control purposes. Examples of the control characters are “null,” “line feed,” “start of text,” and “escape.” The other characters are graphic symbols that can be printed or displayed and include the letters of the alphabet (lowercase and uppercase), the ten decimal digits, punctuation signs, and other commonly used symbols.

Table 2–7 is a listing of the ASCII code showing the decimal, hexadecimal, and binary representations for each character and symbol. The left section of the table lists the names of the 32 control characters (00 through 1F hexadecimal). The graphic symbols are listed in the rest of the table (20 through 7F hexadecimal).

InfoNote

A computer keyboard has a dedicated microprocessor that constantly scans keyboard circuits to detect when a key has been pressed and released. A unique scan code is produced by computer software representing that particular key. The scan code is then converted to an alphanumeric code (ASCII) for use by the computer.

EXAMPLE 2-38

Use Table 2–7 to determine the binary ASCII codes that are entered from the computer’s keyboard when the following C language program statement is typed in. Also express each code in hexadecimal.

if (x > 5)

Solution

The ASCII code for each symbol is found in Table 2–7.

Symbol	Binary	Hexadecimal
i	1101001	69 ₁₆
f	1100110	66 ₁₆
Space	0100000	20 ₁₆
(	0101000	28 ₁₆
x	1111000	78 ₁₆
>	0111110	3E ₁₆
5	0110101	35 ₁₆
)	0101001	29 ₁₆

Related Problem

Use Table 2–7 to determine the sequence of ASCII codes required for the following C program statement and express each code in hexadecimal:

if (y < 8)

The ASCII Control Characters

The first thirty-two codes in the ASCII table (Table 2–7) represent the control characters. These are used to allow devices such as a computer and printer to communicate with each other when passing information and data. The control key function allows a control character to be entered directly from an ASCII keyboard by pressing the control key (CTRL) and the corresponding symbol.

TABLE 2-7

American Standard Code for Information Interchange (ASCII).

Control Characters				Graphic Symbols							
Name	Dec	Binary	Hex	Symbol	Dec	Binary	Hex	Symbol	Dec	Binary	Hex
NUL	0	0000000	00					@	64	1000000	40
SOH	1	0000001	01	space	32	0100000	20	A	65	1000001	41
STX	2	0000010	02	!	33	0100001	21	B	66	1000010	42
ETX	3	0000011	03	"	34	0100010	22	C	67	1000011	43
EOT	4	0000100	04	#	35	0100011	23	D	68	1000100	44
ENQ	5	0000101	05	\$	36	0100100	24	E	69	1000101	45
ACK	6	0000110	06	%	37	0100101	25	F	70	1000110	46
BEL	7	0000111	07	&	38	0100110	26	G	71	1000111	47
BS	8	0001000	08	'	39	0100111	27	H	72	1001000	48
HT	9	0001001	09	(	40	0101000	28	I	73	1001001	49
LF	10	0001010	0A	)	41	0101001	29	J	74	1001010	4A
VT	11	0001011	0B	*	42	0101010	2A	K	75	1001011	4B
FF	12	0001100	0C	+	43	0101011	2B	L	76	1001100	4C
CR	13	0001101	0D	,	44	0101100	2C	M	77	1001101	4D
SO	14	0001110	0E	-	45	0101101	2D	N	78	1001110	4E
SI	15	0001111	0F	.	46	0101110	2E	O	79	1001111	4F
DLE	16	0010000	10	/	47	0101111	2F	P	80	1010000	50
DC1	17	0010001	11	0	48	0110000	30	Q	81	1010001	51
DC2	18	0010010	12	1	49	0110001	31	R	82	1010010	52
DC3	19	0010011	13	2	50	0110010	32	S	83	1010011	53
DC4	20	0010100	14	3	51	0110011	33	T	84	1010100	54
NAK	21	0010101	15	4	52	0110100	34	U	85	1010101	55
SYN	22	0010110	16	5	53	0110101	35	V	86	1010110	56
ETB	23	0010111	17	6	54	0110110	36	W	87	1010111	57
CAN	24	0011000	18	7	55	0110111	37	X	88	1011000	58
EM	25	0011001	19	8	56	0111000	38	Y	89	1011001	59
SUB	26	0011010	1A	9	57	0111001	39	Z	90	1011010	5A
ESC	27	0011011	1B	:	58	0111010	3A	[	91	1011011	5B
FS	28	0011100	1C	;	59	0111011	3B	\	92	1011100	5C
GS	29	0011101	1D	<	60	0111100	3C	]	93	1011101	5D
RS	30	0011110	1E	=	61	0111101	3D	^	94	1011110	5E
US	31	0011111	1F	>	62	0111110	3E	_	95	1011111	5F
				?	63	0111111	3F	Del	127	1111111	7F

Extended ASCII Characters

In addition to the 128 standard ASCII characters, there are an additional 128 characters that were adopted by IBM for use in their PCs (personal computers). Because of the popularity of the PC, these particular extended ASCII characters are also used in applications other than PCs and have become essentially an unofficial standard.

The extended ASCII characters are represented by an 8-bit code series from hexadecimal 80 to hexadecimal FF and can be grouped into the following general categories: foreign (non-English) alphabetic characters, foreign currency symbols, Greek letters, mathematical symbols, drawing characters, bar graphing characters, and shading characters.

Unicode

Unicode provides the ability to encode all of the characters used for the written languages of the world by assigning each character a unique numeric value and name utilizing the universal character set (UCS). It is applicable in computer applications dealing with multi-lingual text, mathematical symbols, or other technical characters.

Unicode has a wide array of characters, and their various encoding forms are used in many environments. While ASCII basically uses 7-bit codes, Unicode uses relatively abstract “code points”—non-negative integer numbers—that map sequences of one or more bytes, using different encoding forms and schemes. To permit compatibility, Unicode assigns the first 128 code points to the same characters as ASCII. One can, therefore, think of ASCII as a 7-bit encoding scheme for a very small subset of Unicode and of the UCS.

Unicode consists of about 100,000 characters, a set of code charts for visual reference, an encoding methodology and set of standard character encodings, and an enumeration of character properties such as uppercase and lowercase. It also consists of a number of related items, such as character properties, rules for text normalization, decomposition, collation, rendering, and bidirectional display order (for the correct display of text containing both right-to-left scripts, such as Arabic or Hebrew, and left-to-right scripts).

SECTION 2-11 CHECKUP

- Convert the following binary numbers to the Gray code:
(a) 1100 (b) 1010 (c) 11010
- Convert the following Gray codes to binary:
(a) 1000 (b) 1010 (c) 11101
- What is the ASCII representation for each of the following characters? Express each as a bit pattern and in hexadecimal notation.
(a) K (b) r (c) \$ (d) +

2-12 Error Codes

In this section, three methods for adding bits to codes to detect a single-bit error are discussed. The parity method of error detection is introduced, and the cyclic redundancy check is discussed. Also, the Hamming code for error detection and correction is presented.

After completing this section, you should be able to

- ◆ Determine if there is an error in a code based on the parity bit
- ◆ Assign the proper parity bit to a code
- ◆ Explain the cyclic redundancy (CRC) check
- ◆ Describe the Hamming code

Parity Method for Error Detection

A parity bit tells if the number of 1s is odd or even.

Many systems use a parity bit as a means for bit **error detection**. Any group of bits contain either an even or an odd number of 1s. A parity bit is attached to a group of bits to make the total number of 1s in a group always even or always odd. An even parity bit makes the total number of 1s even, and an odd parity bit makes the total odd.

A given system operates with even or odd **parity**, but not both. For instance, if a system operates with even parity, a check is made on each group of bits received to make sure the total number of 1s in that group is even. If there is an odd number of 1s, an error has occurred.

As an illustration of how parity bits are attached to a code, Table 2–8 lists the parity bits for each BCD number for both even and odd parity. The parity bit for each BCD number is in the *P* column.

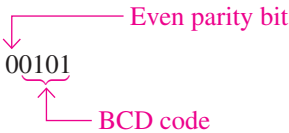
TABLE 2–8
The BCD code with parity bits.

Even Parity		Odd Parity	
<i>P</i>	BCD	<i>P</i>	BCD
0	0000	1	0000
1	0001	0	0001
1	0010	0	0010
0	0011	1	0011
1	0100	0	0100
0	0101	1	0101
0	0110	1	0110
1	0111	0	0111
1	1000	0	1000
0	1001	1	1001

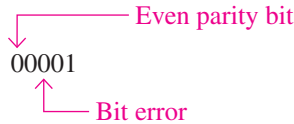
The parity bit can be attached to the code at either the beginning or the end, depending on system design. Notice that the total number of 1s, including the parity bit, is always even for even parity and always odd for odd parity.

Detecting an Error

A parity bit provides for the detection of a single bit error (or any odd number of errors, which is very unlikely) but cannot check for two errors in one group. For instance, let’s assume that we wish to transmit the BCD code 0101. (Parity can be used with any number of bits; we are using four for illustration.) The total code transmitted, including the even parity bit, is



Now let’s assume that an error occurs in the third bit from the left (the 1 becomes a 0).



When this code is received, the parity check circuitry determines that there is only a single 1 (odd number), when there should be an even number of 1s. Because an even number of 1s does not appear in the code when it is received, an error is indicated.

An odd parity bit also provides in a similar manner for the detection of a single error in a given group of bits.

EXAMPLE 2-39

Assign the proper even parity bit to the following code groups:

- (a) 1010 (b) 111000 (c) 101101
 (d) 1000111001001 (e) 101101011111

Solution

Make the parity bit either 1 or 0 as necessary to make the total number of 1s even. The parity bit will be the left-most bit (color).

- (a) **0**1010 (b) **1**111000 (c) **0**101101
 (d) **0**100011100101 (e) **1**01101011111

Related Problem

Add an even parity bit to the 7-bit ASCII code for the letter K.

EXAMPLE 2-40

An odd parity system receives the following code groups: 10110, 11010, 110011, 110101110100, and 1100010101010. Determine which groups, if any, are in error.

Solution

Since odd parity is required, any group with an even number of 1s is incorrect. The following groups are in error: **110011** and **1100010101010**.

Related Problem

The following ASCII character is received by an odd parity system: 00110111. Is it correct?

Cyclic Redundancy Check

The **cyclic redundancy check (CRC)** is a widely used code used for detecting one- and two-bit transmission errors when digital data are transferred on a communication link. The communication link can be between two computers that are connected to a network or between a digital storage device (such as a CD, DVD, or a hard drive) and a PC. If it is properly designed, the CRC can also detect multiple errors for a number of bits in sequence (burst errors). In CRC, a certain number of check bits, sometimes called a *checksum*, are appended to the data bits (added to end) that are being transmitted. The transmitted data are tested by the receiver for errors using the CRC. Not every possible error can be identified, but the CRC is much more efficient than just a simple parity check.

CRC is often described mathematically as the division of two polynomials to generate a remainder. A polynomial is a mathematical expression that is a sum of terms with positive exponents. When the coefficients are limited to 1s and 0s, it is called a *univariate polynomial*. An example of a univariate polynomial is $1x^3 + 0x^2 + 1x^1 + 1x^0$ or simply $x^3 + x^1 + x^0$, which can be fully described by the 4-bit binary number 1011. Most cyclic redundancy checks use a 16-bit or larger polynomial, but for simplicity the process is illustrated here with four bits.

Modulo-2 Operations

Simply put, CRC is based on the division of two binary numbers; and, as you know, division is just a series of subtractions and shifts. To do subtraction, a method called *modulo-2* addition can be used. Modulo-2 addition (or subtraction) is the same as binary addition with the carries discarded, as shown in the truth table in Table 2-9. **Truth tables** are widely used to describe the operation of logic circuits, as you will learn in Chapter 3. With two bits, there is a total of four possible combinations, as shown in the table. This particular table describes the modulo-2 operation also known as *exclusive-OR* and can be implemented with a logic

TABLE 2-9

Modulo-2 operation.

Input Bits	Output Bit
0 0	0
0 1	1
1 0	1
1 1	0

gate that will be introduced in Chapter 3. A simple rule for modulo-2 is that the output is 1 if the inputs are different; otherwise, it is 0.

CRC Process

The process is as follows:

1. Select a fixed generator code; it can have fewer bits than the data bits to be checked. This code is understood in advance by both the sending and receiving devices and must be the same for both.
2. Append a number of 0s equal to the number of bits in the generator code to the data bits.
3. Divide the data bits including the appended bits by the generator code bits using modulo-2.
4. If the remainder is 0, the data and appended bits are sent as is.
5. If the remainder is not 0, the appended bits are made equal to the remainder bits in order to get a 0 remainder before data are sent.
6. At the receiving end, the receiver divides the incoming appended data bit code by the same generator code as used by the sender.
7. If the remainder is 0, there is no error detected (it is possible in rare cases for multiple errors to cancel). If the remainder is not 0, an error has been detected in the transmission and a retransmission is requested by the receiver.

Figure 2–8 illustrates the CRC process.

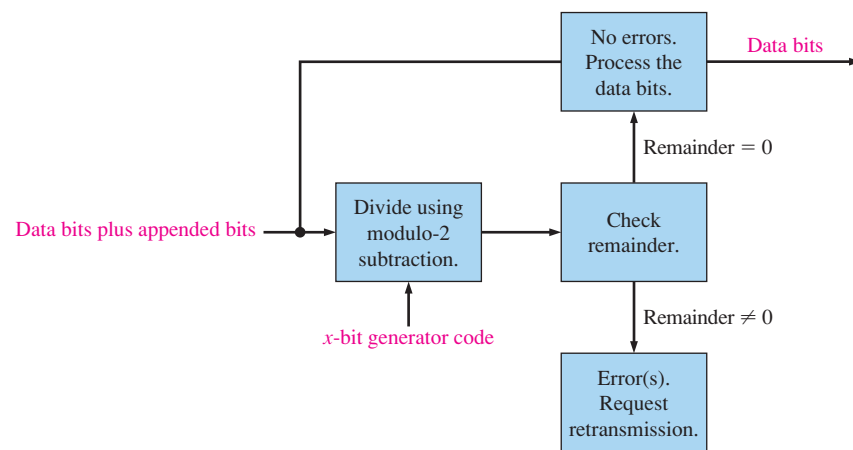
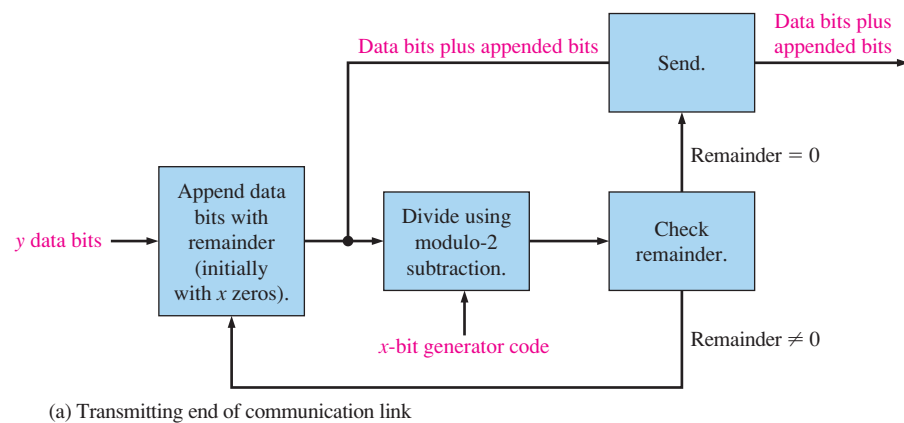


FIGURE 2–8 The CRC process.

EXAMPLE 2-41

Determine the transmitted CRC for the following byte of data (D) and generator code (G). Verify that the remainder is 0.

D: 11010011

G: 1010

Solution

Since the generator code has four data bits, add four 0s (blue) to the data byte. The appended data (D') is

$$D' = 110100110000$$

Divide the appended data by the generator code (red) using the modulo-2 operation until all bits have been used.

$$\begin{array}{r} D' \\ G \end{array} = \frac{110100110000}{1010}$$

```

110100110000
1010
---
1110
1010
---
1000
1010
---
1011
1010
---
1000
1010
---
100

```

Remainder = 0100. Since the remainder is not 0, append the data with the four remainder bits (blue). Then divide by the generator code (red). The transmitted CRC is **110100110100**.

```

110100110100
1010
---
1110
1010
---
1000
1010
---
1011
1010
---
1010
1010
---
00

```

Remainder = 0

Related Problem

Change the generator code to 1100 and verify that a 0 remainder results when the CRC process is applied to the data byte (11010011).

EXAMPLE 2-42

During transmission, an error occurs in the second bit from the left in the appended data byte generated in Example 2-41. The received data is

$$D' = 100100110100$$

Apply the CRC process to the received data to detect the error using the same generator code (1010).

Solution

$$\begin{array}{r}
 100100110100 \\
 \underline{1010} \\
 1100 \\
 \underline{1010} \\
 1101 \\
 \underline{1010} \\
 1111 \\
 \underline{1010} \\
 1010 \\
 \underline{1010} \\
 0100
 \end{array}$$

Remainder = 0100. Since it is not zero, an error is indicated.

Related Problem

Assume two errors in the data byte as follows: 10011011. Apply the CRC process to check for the errors using the same received data and the same generator code.

Hamming Code

The **Hamming code** is used to detect and correct a single-bit error in a transmitted code. To accomplish this, four redundancy bits are introduced in a 7-bit group of data bits. These redundancy bits are interspersed at bit positions 2^n ($n = 0, 1, 2, 3$) within the original data bits. At the end of the transmission, the redundancy bits have to be removed from the data bits. A recent version of the Hamming code places all the redundancy bits at the end of the data bits, making their removal easier than that of the interspersed bits. *A coverage of the classic Hamming code is available on the website.*

SECTION 2-12 CHECKUP

- Which odd-parity code is in error?
(a) 1011 (b) 1110 (c) 0101 (d) 1000
- Which even-parity code is in error?
(a) 11000110 (b) 00101000 (c) 10101010 (d) 11111011
- Add an even parity bit to the end of each of the following codes.
(a) 1010100 (b) 0100000 (c) 1110111 (d) 1000110
- What does CRC stand for?
- Apply modulo-2 operations to determine the following:
(a) $1 + 1$ (b) $1 - 1$ (c) $1 - 0$ (d) $0 + 1$

SUMMARY

- A binary number is a weighted number in which the weight of each whole number digit is a positive power of two and the weight of each fractional digit is a negative power of two. The whole number weights increase from right to left—from least significant digit to most significant.
- A binary number can be converted to a decimal number by summing the decimal values of the weights of all the 1s in the binary number.
- A decimal whole number can be converted to binary by using the sum-of-weights or the repeated division-by-2 method.
- A decimal fraction can be converted to binary by using the sum-of-weights or the repeated multiplication-by-2 method.
- The basic rules for binary addition are as follows:

$$0 + 0 = 0$$

$$0 + 1 = 1$$

$$1 + 0 = 1$$

$$1 + 1 = 10$$

- The basic rules for binary subtraction are as follows:

$$0 - 0 = 0$$

$$1 - 1 = 0$$

$$1 - 0 = 1$$

$$10 - 1 = 1$$

- The 1's complement of a binary number is derived by changing 1s to 0s and 0s to 1s.
- The 2's complement of a binary number can be derived by adding 1 to the 1's complement.
- Binary subtraction can be accomplished with addition by using the 1's or 2's complement method.
- A positive binary number is represented by a 0 sign bit.
- A negative binary number is represented by a 1 sign bit.
- For arithmetic operations, negative binary numbers are represented in 1's complement or 2's complement form.
- In an addition operation, an overflow is possible when both numbers are positive or when both numbers are negative. An incorrect sign bit in the sum indicates the occurrence of an overflow.
- The hexadecimal number system consists of 16 digits and characters, 0 through 9 followed by A through F.
- One hexadecimal digit represents a 4-bit binary number, and its primary usefulness is in simplifying bit patterns and making them easier to read.
- A decimal number can be converted to hexadecimal by the repeated division-by-16 method.
- The octal number system consists of eight digits, 0 through 7.
- A decimal number can be converted to octal by using the repeated division-by-8 method.
- Octal-to-binary conversion is accomplished by simply replacing each octal digit with its 3-bit binary equivalent. The process is reversed for binary-to-octal conversion.
- A decimal number is converted to BCD by replacing each decimal digit with the appropriate 4-bit binary code.
- The ASCII is a 7-bit alphanumeric code that is used in computer systems for input and output of information.
- A parity bit is used to detect an error in a code.
- The CRC (cyclic redundancy check) is based on polynomial division using modulo-2 operations.

KEY TERMS

Key terms and other bold terms in the chapter are defined in the end-of-book glossary.

Alphanumeric Consisting of numerals, letters, and other characters.

ASCII American Standard Code for Information Interchange; the most widely used alphanumeric code.

BCD Binary coded decimal; a digital code in which each of the decimal digits, 0 through 9, is represented by a group of four bits.

Byte A group of eight bits.

Cyclic redundancy check (CRC) A type of error detection code.

Floating-point number A number representation based on scientific notation in which the number consists of an exponent and a mantissa.

Hexadecimal Describes a number system with a base of 16.

LSB Least significant bit; the right-most bit in a binary whole number or code.

MSB Most significant bit; the left-most bit in a binary whole number or code.

Octal Describes a number system with a base of eight.

Parity In relation to binary codes, the condition of evenness or oddness of the number of 1s in a code group.

TRUE/FALSE QUIZ

Answers are at the end of the chapter.

5. The 1's complement is found by inverting all bits.

7. The left-most bit (the MSB) in a signed binary number is the sign bit. The right-most bit is the LSB (Least Significant Bit).

10. A single-bit error can be detected using an odd or even parity bit.

- ✓ 1. The octal number system is a weighted system with eight digits.
- ✓ 2. The binary number system is a weighted system with two digits.
- ✓ 3. MSB stands for most significant bit.
4. In hexadecimal, $9 + 1 = 10$. **$9 + 1 = A$**
- ✓ 5. The 1's complement of the binary number 1010 is 0101.
6. The 2's complement of the binary number 1111 is 0000. **6. The 2's complement of 1111 is 0001. 1's complement is 0000 then plus 1 give 0001.**
7. The right-most bit in a signed binary number is the sign bit.
- ✓ 8. The hexadecimal number system has 16 characters, six of which are alphabetic characters. **0-9, A-F**
- ✓ 9. BCD stands for binary coded decimal.
- ✓ 10. An error in a given code can be detected by verifying the parity bit.
- ✓ 11. CRC stands for cyclic redundancy check.
12. The modulo-2 sum of 11 and 10 is 100. **Modulo-2 sum is the XOR operation. $11 \text{ XOR } 10 = 01$**

SELF-TEST

Answers are at the end of the chapter.

1. $3 \times 10^1 + 4 \times 10^0$ is
(a) 0.34 (b) 3.4 (c) **34** (d) 340
2. The decimal equivalent of 1000 is
(a) 2 (b) 4 (c) 6 (d) **8**
3. The binary number 11011101 is equal to the decimal number
(a) 121 (b) **221** (c) 441 (d) 256
4. The decimal number 21 is equivalent to the binary number
(a) **10101** (b) 10001 (c) 10000 (d) 11111
5. The decimal number 250 is equivalent to the binary number
(a) **11111010** (b) 11110110 (c) 11111000 (d) 11111011
6. The sum of $1111 + 1111$ in binary equals
(a) 0000 (b) 2222 (c) **11110** (d) 11111

7. The difference of $1000 - 100$ equals
(a) 100 (b) 101 (c) 110 (d) 111
8. The 1's complement of 11110000 is
(a) 11111111 (b) 11111110 (c) 00001111 (d) 10000001
9. The 2's complement of 11001100 is
(a) 00110011 (b) 00110100 (c) 00110101 (d) 00110110
10. The decimal number +122 is expressed in the 2's complement form as
(a) 01111010 (b) 11111010 (c) 01000101 (d) 10000101
11. The decimal number -34 is expressed in the 2's complement form as
(a) 01011110 (b) 10100010 (c) 11011110 (d) 01011101
12. A single-precision floating-point binary number has a total of
(a) 8 bits (b) 16 bits (c) 24 bits (d) 32 bits
13. In the 2's complement form, the binary number 10010011 is equal to the decimal number
(a) -19 (b) +109 (c) +91 (d) -109
14. The binary number 101100111001010100001 can be written in octal as
(a) 5471230₈ (b) 5471241₈ (c) 2634521₈ (d) 23162501₈
15. The binary number 10001101010001101111 can be written in hexadecimal as
(a) AD467₁₆ (b) 8C46F₁₆ (c) 8D46F₁₆ (d) AE46F₁₆
16. The binary number for F7A9₁₆ is
(a) 1111011110101001 (b) 1110111110101001
(c) 1111111010110001 (d) 1111011010101001
17. The BCD number for decimal 473 is
(a) 111011010 (b) 110001110011 (c) 010001110011 (d) 010011110011
18. Refer to Table 2-7. The command STOP in ASCII is
(a) 10100111010100100011111010000 (b) 1010010100110010011101010000
(c) 1001010110110110011101010001 (d) 1010011101010010011101100100
19. The code that has an even-parity error is
(a) 1010011 (b) 1101000 (c) 1001000 (d) 1110111
20. In the cyclic redundancy check, the absence of errors is indicated by
(a) Remainder = generator code (b) Remainder = 0
(c) Remainder = 1 (d) Quotient = 0

PROBLEMS

Answers to odd-numbered problems are at the end of the book.

Section 2-1 Decimal Numbers

1. What is the weight of 7 in each of the following decimal numbers?
(a) 1947 (b) 1799 (c) 1979
2. Express each of the following decimal numbers as a power of ten:
(a) 1000 (b) 10000000 (c) 1000000000
3. Give the value of each digit in the following decimal numbers:
(a) 263 (b) 5436 (c) 234543
4. How high can you count with six decimal digits?

Section 2-2 Binary Numbers

5. Convert the following binary numbers to decimal:
(a) 001 (b) 010 (c) 101 (d) 110
(e) 1010 (f) 1011 (g) 1110 (h) 1111
6. Convert the following binary numbers into decimal:
(a) 100001 (b) 100111 (c) 101010 (d) 111001
(e) 1100000 (f) 11111101 (g) 11110010 (h) 11111111

7. Convert each binary number to decimal:

- | | | |
|------------------|-------------------|------------------|
| (a) 110011.11 | (b) 101010.01 | (c) 1000001.111 |
| (d) 1111000.101 | (e) 1011100.10101 | (f) 1110001.0001 |
| (g) 1011010.1010 | (h) 1111111.11111 | |

8. What is the highest decimal number that can be represented by each of the following numbers of binary digits (bits)?

- | | | | | |
|-----------|-----------|----------|----------|------------|
| (a) two | (b) three | (c) four | (d) five | (e) six |
| (f) seven | (g) eight | (h) nine | (i) ten | (j) eleven |

9. How many bits are required to represent the following decimal numbers?

- | | | | |
|---------|---------|---------|---------|
| (a) 5 | (b) 10 | (c) 15 | (d) 20 |
| (e) 100 | (f) 120 | (g) 140 | (h) 160 |

10. Generate the binary sequence for each decimal sequence:

- | | | |
|-------------------|-------------------|-------------------|
| (a) 0 through 7 | (b) 8 through 15 | (c) 16 through 31 |
| (d) 32 through 63 | (e) 64 through 75 | |

Section 2-3 Decimal-to-Binary Conversion

11. Convert each decimal number to binary by using the sum-of-weights method:

- | | | | |
|--------|--------|---------|---------|
| (a) 12 | (b) 15 | (c) 25 | (d) 50 |
| (e) 65 | (f) 97 | (g) 127 | (h) 198 |

12. Convert each decimal fraction to binary using the sum-of-weights method:

- | | | |
|----------|-----------|------------|
| (a) 0.26 | (b) 0.762 | (c) 0.0975 |
|----------|-----------|------------|

13. Convert each decimal number to binary using repeated division by 2:

- | | | | |
|--------|--------|--------|--------|
| (a) 13 | (b) 17 | (c) 23 | (d) 30 |
| (e) 35 | (f) 40 | (g) 49 | (h) 60 |

14. Convert each decimal fraction to binary using repeated multiplication by 2:

- | | | |
|----------|-----------|------------|
| (a) 0.76 | (b) 0.456 | (c) 0.8732 |
|----------|-----------|------------|

Section 2-4 Binary Arithmetic

15. Add the binary numbers:

- | | | |
|-----------------|------------------|-------------------|
| (a) $10 + 10$ | (b) $10 + 11$ | (c) $100 + 11$ |
| (d) $111 + 101$ | (e) $1111 + 111$ | (f) $1111 + 1111$ |

16. Use direct subtraction on the following binary numbers:

- | | | |
|-----------------|------------------|---------------------|
| (a) $10 - 1$ | (b) $100 - 11$ | (c) $110 - 100$ |
| (d) $1111 - 11$ | (e) $1101 - 101$ | (f) $110000 - 1111$ |

17. Perform the following binary multiplications:

- | | | |
|-----------------------|------------------------|------------------------|
| (a) 11×10 | (b) 101×11 | (c) 111×110 |
| (d) 1100×101 | (e) 1110×1110 | (f) 1111×1100 |

18. Divide the binary numbers as indicated:

- | | | |
|-------------------|--------------------|---------------------|
| (a) $110 \div 11$ | (b) $1010 \div 10$ | (c) $1111 \div 101$ |
|-------------------|--------------------|---------------------|

Section 2-5 Complements of Binary Numbers

19. What are two ways of representing zero in 1's complement form?

20. How is zero represented in 2's complement form?

21. Determine the 1's complement of each binary number:

- | | | |
|--------------|-------------|--------------|
| (a) 100 | (b) 111 | (c) 1100 |
| (d) 10111011 | (e) 1001010 | (f) 10101010 |

22. Determine the 2's complement of each binary number using either method:

- | | | | |
|------------|-----------|--------------|--------------|
| (a) 11 | (b) 110 | (c) 1010 | (d) 1001 |
| (e) 101010 | (f) 11001 | (g) 11001100 | (h) 11000111 |

Section 2-6 Signed Numbers

23. Express each decimal number in binary as an 8-bit sign-magnitude number:
(a) +29 (b) -85 (c) +100 (d) -123
24. Express each decimal number as an 8-bit number in the 1's complement form:
(a) -34 (b) +57 (c) -99 (d) +115
25. Express each decimal number as an 8-bit number in the 2's complement form:
(a) +12 (b) -68 (c) +101 (d) -125
26. Determine the decimal value of each signed binary number in the sign-magnitude form:
(a) 10011001 (b) 01110100 (c) 10111111
27. Determine the decimal value of each signed binary number in the 1's complement form:
(a) 10011001 (b) 01110100 (c) 10111111
28. Determine the decimal value of each signed binary number in the 2's complement form:
(a) 10011001 (b) 01110100 (c) 10111111
29. Express each of the following sign-magnitude binary numbers in single-precision floating-point format:
(a) 0111110000101011 (b) 100110000011000
30. Determine the values of the following single-precision floating-point numbers:
(a) 1 10000001 01001001110001000000000
(b) 0 11001100 10000111110100100000000

Section 2-7 Arithmetic Operations with Signed Numbers

31. Convert each pair of decimal numbers to binary and add using the 2's complement form:
(a) 33 and 15 (b) 56 and -27 (c) -46 and 25 (d) -110 and -84
32. Perform each addition in the 2's complement form:
(a) 00010110 + 00110011 (b) 01110000 + 10101111
33. Perform each addition in the 2's complement form:
(a) 10001100 + 00111001 (b) 11011001 + 11100111
34. Perform each subtraction in the 2's complement form:
(a) 00110011 - 00010000 (b) 01100101 - 11101000
35. Multiply 01101010 by 11110001 in the 2's complement form.
36. Divide 10001000 by 00100010 in the 2's complement form.

Section 2-8 Hexadecimal Numbers

37. Convert each hexadecimal number to binary:
(a) 46_{16} (b) 54_{16} (c) $B4_{16}$ (d) $1A3_{16}$
(e) FA_{16} (f) ABC_{16} (g) $ABCD_{16}$
38. Convert each binary number to hexadecimal:
(a) 1111 (b) 1011 (c) 11111
(d) 10101010 (e) 10101100 (f) 10111011
39. Convert each hexadecimal number to decimal:
(a) 42_{16} (b) 64_{16} (c) $2B_{16}$ (d) $4D_{16}$
(e) FF_{16} (f) BC_{16} (g) $6F1_{16}$ (h) ABC_{16}
40. Convert each decimal number to hexadecimal:
(a) 10 (b) 15 (c) 32 (d) 54
(e) 365 (f) 3652 (g) 7825 (h) 8925
41. Perform the following additions:
(a) $25_{16} + 33_{16}$ (b) $43_{16} + 62_{16}$ (c) $A4_{16} + F5_{16}$ (d) $FC_{16} + AE_{16}$
42. Perform the following subtractions:
(a) $60_{16} - 39_{16}$ (b) $A5_{16} - 98_{16}$ (c) $F1_{16} - A6_{16}$ (d) $AC_{16} - 10_{16}$

Section 2-9 Octal Numbers

43. Convert each octal number to decimal:

- (a) 14_8 (b) 53_8 (c) 67_8 (d) 174_8
 (e) 635_8 (f) 254_8 (g) 2673_8 (h) 7777_8

44. Convert each decimal number to octal by repeated division by 8:

- (a) 23 (b) 45 (c) 65 (d) 84
 (e) 124 (f) 156 (g) 654 (h) 9999

45. Convert each octal number into binary:

- (a) 17_8 (b) 26_8 (c) 145_8 (d) 456_8
 (e) 653_8 (f) 777_8

46. Convert each binary number to octal:

- (a) 100 (b) 110 (c) 1100
 (d) 1111 (e) 11001 (f) 11110
 (g) 110011 (h) 101010 (i) 10101111

Section 2-10 Binary Coded Decimal (BCD)

47. Convert each of the following decimal numbers to 8421 BCD:

- (a) 10 (b) 13 (c) 18 (d) 21 (e) 25 (f) 36
 (g) 44 (h) 57 (i) 69 (j) 98 (k) 125 (l) 156

48. Convert each of the decimal numbers in Problem 47 to straight binary, and compare the number of bits required with that required for BCD.

49. Convert the following decimal numbers to BCD:

- (a) 104 (b) 128 (c) 132 (d) 150 (e) 186
 (f) 210 (g) 359 (h) 547 (i) 1051

50. Convert each of the BCD numbers to decimal:

- (a) 0001 (b) 0110 (c) 1001
 (d) 00011000 (e) 00011001 (f) 00110010
 (g) 01000101 (h) 10011000 (i) 100001110000

51. Convert each of the BCD numbers to decimal:

- (a) 10000000 (b) 001000110111
 (c) 001101000110 (d) 010000100001
 (e) 011101010100 (f) 100000000000
 (g) 100101111000 (h) 0001011010000011
 (i) 100100000011000 (j) 0110011001100111

52. Add the following BCD numbers:

- (a) $0010 + 0001$ (b) $0101 + 0011$
 (c) $0111 + 0010$ (d) $1000 + 0001$
 (e) $00011000 + 00010001$ (f) $01100100 + 00110011$
 (g) $01000000 + 01000111$ (h) $10000101 + 00010011$

53. Add the following BCD numbers:

- (a) $1000 + 0110$ (b) $0111 + 0101$
 (c) $1001 + 1000$ (d) $1001 + 0111$
 (e) $00100101 + 00100111$ (f) $01010001 + 01011000$
 (g) $10011000 + 10010111$ (h) $010101100001 + 011100001000$

54. Convert each pair of decimal numbers to BCD, and add as indicated:

- (a) $4 + 3$ (b) $5 + 2$ (c) $6 + 4$ (d) $17 + 12$
 (e) $28 + 23$ (f) $65 + 58$ (g) $113 + 101$ (h) $295 + 157$

Section 2-11 Digital Codes

55. In a certain application a 4-bit binary sequence cycles from 1111 to 0000 periodically. There are four bit changes, and because of circuit delays, these changes may not occur at the same

instant. For example, if the LSB changes first, the number will appear as 1110 during the transition from 1111 to 0000 and may be misinterpreted by the system. Illustrate how the Gray code avoids this problem.

56. Convert each binary number to Gray code:

(a) 11011 (b) 1001010 (c) 1111011101110

57. Convert each Gray code to binary:

(a) 1010 (b) 00010 (c) 11000010001

58. Convert each of the following decimal numbers to ASCII. Refer to Table 2–7.

(a) 1 (b) 3 (c) 6 (d) 10 (e) 18
(f) 29 (g) 56 (h) 75 (i) 107

59. Determine each ASCII character. Refer to Table 2–7.

(a) 0011000 (b) 1001010 (c) 0111101
(d) 0100011 (e) 0111110 (f) 1000010

60. Decode the following ASCII coded message:

```
1001000 1100101 1101100 1101100 1101111 0101110
0100000 1001000 1101111 1110111 0100000 1100001
1110010 1100101 0100000 1111001 1101111 1110101
0111111
```

61. Write the message in Problem 60 in hexadecimal.

62. Convert the following statement to ASCII:

30 INPUT A, B

Section 2–12 Error Codes

63. Determine which of the following even parity codes are in error:

(a) 100110010 (b) 011101010 (c) 10111111010001010

64. Determine which of the following odd parity codes are in error:

(a) 11110110 (b) 00110001 (c) 01010101010101010

65. Attach the proper even parity bit to each of the following bytes of data:

(a) 10100100 (b) 00001001 (c) 11111110

66. Apply modulo-2 to the following:

(a) $1100 + 1011$ (b) $1111 + 0100$ (c) $10011001 + 100011100$

67. Verify that modulo-2 subtraction is the same as modulo-2 addition by adding the result of each operation in problem 66 to either of the original numbers to get the other number. This will show that the result is the same as the difference of the two numbers.

68. Apply CRC to the data bits 10110010 using the generator code 1010 to produce the transmitted CRC code.

69. Assume that the code produced in problem 68 incurs an error in the most significant bit during transmission. Apply CRC to detect the error.

ANSWERS

SECTION CHECKUPS

Section 2–1 Decimal Numbers

- (a) 1370: 10 (b) 6725: 100 (c) 7051: 1000 (d) 58.72: 0.1
- (a) $51 = (5 \times 10) + (1 \times 1)$
(b) $137 = (1 \times 100) + (3 \times 10) + (7 \times 1)$
(c) $1492 = (1 \times 1000) + (4 \times 100) + (9 \times 10) + (2 \times 1)$
(d) $106.58 = (1 \times 100) + (0 \times 10) + (6 \times 1) + (5 \times 0.1) + (8 \times 0.01)$

Section 2-2 Binary Numbers

- $2^8 - 1 = 255$
- Weight is 16.
- $10111101.011 = 189.375$

Section 2-3 Decimal-to-Binary Conversion

- (a) $23 = 10111$ (b) $57 = 111001$ (c) $45.5 = 101101.1$
- (a) $14 = 1110$ (b) $21 = 10101$ (c) $0.375 = 0.011$

Section 2-4 Binary Arithmetic

- (a) $1101 + 1010 = 10111$ (b) $10111 + 01101 = 100100$
- (a) $1101 - 0100 = 1001$ (b) $1001 - 0111 = 0010$
- (a) $110 \times 111 = 101010$ (b) $1100 \div 011 = 100$

Section 2-5 Complements of Binary Numbers

- (a) 1's comp of $00011010 = 11100101$ (b) 1's comp of $11110111 = 00001000$
(c) 1's comp of $10001101 = 01110010$
- (a) 2's comp of $00010110 = 11101010$ (b) 2's comp of $11111100 = 00000100$
(c) 2's comp of $10010001 = 01101111$

Section 2-6 Signed Numbers

- Sign-magnitude: $+9 = 00001001$
- 1's comp: $-33 = 11011110$
- 2's comp: $-46 = 11010010$
- Sign bit, exponent, and mantissa

Section 2-7 Arithmetic Operations with Signed Numbers

- Cases of addition: positive number is larger, negative number is larger, both are positive, both are negative
- $00100001 + 10111100 = 11011101$
- $01110111 - 00110010 = 01000101$
- Sign of product is positive.
- $00000101 \times 01111111 = 01001111011$
- Sign of quotient is negative.
- $00110000 \div 00001100 = 00000100$

Section 2-8 Hexadecimal Numbers

- (a) $10110011 = B3_{16}$ (b) $110011101000 = CE8_{16}$
- (a) $57_{16} = 01010111$ (b) $3A5_{16} = 001110100101$
(c) $F80B_{16} = 1111100000001011$
- $9B30_{16} = 39,728_{10}$
- $573_{10} = 23D_{16}$
- (a) $18_{16} + 34_{16} = 4C_{16}$ (b) $3F_{16} + 2A_{16} = 69_{16}$
- (a) $75_{16} - 21_{16} = 54_{16}$ (b) $94_{16} - 5C_{16} = 38_{16}$

Section 2-9 Octal Numbers

- (a) $73_8 = 59_{10}$ (b) $125_8 = 85_{10}$
- (a) $98_{10} = 142_8$ (b) $163_{10} = 243_8$

3. (a) $46_8 = 100110$ (b) $723_8 = 111010011$ (c) $5624_8 = 101110010100$
 4. (a) $110101111 = 657_8$ (b) $1001100010 = 1142_8$ (c) $10111111001 = 2771_8$

Section 2-10 Binary Coded Decimal (BCD)

1. (a) 0010: 2 (b) 1000: 8 (c) 0001: 1 (d) 0100: 4
 2. (a) $6_{10} = 0110$ (b) $15_{10} = 00010101$ (c) $273_{10} = 001001110011$
 (d) $849_{10} = 100001001001$
 3. (a) $10001001 = 89_{10}$ (b) $001001111000 = 278_{10}$ (c) $000101010111 = 157_{10}$
 4. A 4-bit sum is invalid when it is greater than 9_{10} .

Section 2-11 Digital Codes

1. (a) $1100_2 = 1010$ Gray (b) $1010_2 = 1111$ Gray (c) $11010_2 = 10111$ Gray
 2. (a) 1000 Gray = 1111_2 (b) 1010 Gray = 1100_2 (c) 11101 Gray = 10110_2
 3. (a) K: $1001011 \rightarrow 4B_{16}$ (b) r: $1110010 \rightarrow 72_{16}$
 (c) \$: $0100100 \rightarrow 24_{16}$ (d) +: $0101011 \rightarrow 2B_{16}$

Section 2-12 Error Codes

1. (c) 0101 has an error.
 2. (d) 11111011 has an error.
 3. (a) 1010100**1** (b) 0100000**1** (c) 1110111**0** (d) 1000110**1**
 4. Cyclic redundancy check
 5. (a) 0 (b) 0 (c) 1 (d) 1

RELATED PROBLEMS FOR EXAMPLES

- 2-1 9 has a value of 900, 3 has a value of 30, 9 has a value of 9.
 2-2 6 has a value of 60, 7 has a value of 7, 9 has a value of 9/10 (0.9), 2 has a value of 2/100 (0.02), 4 has a value of 4/1000 (0.004).
 2-3 $10010001 = 128 + 16 + 1 = 145$
 2-4 $10.111 = 2 + 0.5 + 0.25 + 0.125 = 2.875$
 2-5 $125 = 64 + 32 + 16 + 8 + 4 + 1 = 1111101$
 2-6 $39 = 100111$
 2-7 $1111 + 1100 = 11011$
 2-8 $111 - 100 = 011$
 2-9 $110 - 101 = 001$
 2-10 $1101 \times 1010 = 10000010$
 2-11 $1100 \div 100 = 11$
 2-12 00110101
 2-13 01000000
 2-14 See Table 2-10.

TABLE 2-10

	Sign-Magnitude	1's Comp	2's Comp
+19	00010011	00010011	00010011
-19	10010011	11101100	11101101

- 2-15 $01110111 = +119_{10}$
 2-16 $11101011 = -20_{10}$
 2-17 $11010111 = -41_{10}$

2-18 11000010001010011000000000

2-19 01010101

2-20 00010001

2-21 1001000110

2-22 $(83)(-59) = -4897$ (10110011011111 in 2's comp)

2-23 $100 \div 25 = 4$ (0100)

2-24 $4F79C_{16}$

2-25 0110101111010011₂

2-26 $6BD_{16} = 011010111101 = 2^{10} + 2^9 + 2^7 + 2^5 + 2^4 + 2^3 + 2^2 + 2^0$
 $= 1024 + 512 + 128 + 32 + 16 + 8 + 4 + 1 = 1725_{10}$

2-27 $60A_{16} = (6 \times 256) + (0 \times 16) + (10 \times 1) = 1546_{10}$

2-28 $2591_{10} = A1F_{16}$

2-29 $4C_{16} + 3A_{16} = 86_{16}$

2-30 $BCD_{16} - 173_{16} = A5A_{16}$

2-31 (a) $001011_2 = 11_{10} = 13_8$ (b) $010101_2 = 21_{10} = 25_8$
 (c) $001100000_2 = 96_{10} = 140_8$ (d) $111101010110_2 = 3926_{10} = 7526_8$

2-32 1250762₈

2-33 1001011001110011

2-34 82,276₁₀

2-35 1001100101101000

2-36 10000010

2-37 (a) 111011 (Gray) (b) 111010₂

2-38 The sequence of codes for if ($y < 8$) is $69_{16}66_{16}20_{16}28_{16}79_{16}3C_{16}38_{16}29_{16}$

2-39 01001011

2-40 Yes

2-41 A 0 remainder results

2-42 Errors are indicated.

TRUE/FALSE QUIZ

1. T 2. T 3. T 4. F 5. T 6. F 7. F 8. T 9. T 10. T
 11. T 12. F

SELF-TEST

1. (c) 2. (d) 3. (b) 4. (a) 5. (a) 6. (c) 7. (a) 8. (c)
 9. (b) 10. (a) 11. (c) 12. (d) 13. (d) 14. (b) 15. (c) 16. (a)
 17. (c) 18. (a) 19. (b) 20. (b)